Universidad Mayor de San Andrés
Facultad de Ciencias Puras y Naturales
Carrera de Informática

UMSA

CRIPTOGRAFÍA DE RETÍCULOS MATEMÁTICOS

Fundamentos Teóricos, Estándares NIST FIPS 203/204 y Aplicación al Protocolo de Votación Electrónica Post-Cuántica

---

Un texto didáctico y autocontenido para comprender desde la geometría euclidiana de retículos hasta los algoritmos ML-KEM-768, ML-DSA-65 y su validación experimental

Modalidad:	Proyecto de Grado / Tesis de Licenciatura
Metodología:	Design Science Research (DSR)
Estándares:	NIST FIPS 203 (ML-KEM) / NIST FIPS 204 (ML-DSA)
Lugar y Año:	La Paz, Bolivia — 2026

"En la geometría de los espacios discretos de alta dimensión reside la fortaleza que preserva la privacidad del sufragio democrático ante la revolución cuántica."

Índice general

Prefacio **vii**

I Fundamentos Matemáticos y Criptográficos **1**

1. El Desafío Cuántico a la Criptografía Clásica **3**
1.1. Historia breve de la criptografía de clave pública . . . 3
1.1.1. Por qué estos sistemas están en riesgo . . . 3
1.2. La respuesta: Criptografía Post-Cuántica . . . 4

2. Retículos: Definiciones y Geometría **5**
2.1. ¿Qué es un retículo? . . . 5
2.1.1. Ejemplos básicos . . . 5
2.2. Propiedades geométricas fundamentales . . . 5
2.2.1. Determinante y volumen fundamental . . . 5
2.2.2. Primer mínimo . . . 6
2.3. Problemas difíciles en retículos . . . 6
2.3.1. SVP — Problema del Vector Más Corto . . . 6
2.3.2. CVP — Problema del Vector Más Cercano . . . 6
2.3.3. SIS — Short Integer Solution . . . 6

3. Learning With Errors (LWE) y sus Variantes **9**
3.1. El Problema LWE . . . 9
3.1.1. Intuición: por qué LWE es difícil . . . 9
3.2. Ring-LWE (RLWE) . . . 10
3.2.1. Ventajas de R_q . . . 10
3.3. Module-LWE (MLWE) — La base de ML-KEM y ML-DSA . . . 10
3.3.1. Parámetros de ML-KEM-768 . . . 11
3.3.2. La Transformada NTT . . . 11

4. Esquemas de Encapsulamiento de Clave (KEM) **13**
4.1. Concepto de KEM . . . 13
4.2. Construcción de ML-KEM (FIPS 203) . . . 13
4.2.1. Generación de claves . . . 14
4.2.2. Encapsulamiento . . . 14
4.2.3. Desencapsulamiento . . . 14
4.2.4. Por qué el desencapsulamiento funciona . . . 14

5. Firmas Digitales Basadas en Retículos **17**
5.1. Concepto de firma digital . . . 17
5.2. Construcción de ML-DSA (FIPS 204) — Dilithium . . . 17
5.2.1. Idea intuitiva . . . 17

5.2.2. Generación de claves ML-DSA-65 . . . 18
5.2.3. Firma . . . 18
5.2.4. Verificación . . . 18
5.2.5. Parámetros de ML-DSA-65 . . . 19
5.3. Comparación con ECDSA P-256 . . . 19

II El Protocolo de Votación Electrónica 21

6. Diseño Formal del Protocolo 23
6.1. Motivación: por qué PQC en votación . . . 23
6.2. Entidades del protocolo . . . 23
6.3. Parámetros criptográficos del protocolo . . . 24
6.4. Fases del protocolo . . . 24
6.4.1. Fase (a) — Configuración y generación de claves . . . 24
6.4.2. Fase (b) — Registro y autenticación del Votante . . . 24
6.4.3. Fase (c) — Cifrado del voto y firma de la boleta . . . 24
6.4.4. Fase (d) — Envío al Servidor y verificación . . . 25
6.4.5. Fase (e) — Registro del voto cifrado . . . 25
6.4.6. Fase (f) — Desencapsulamiento y conteo final . . . 25
6.5. Estructuras de datos . . . 25
6.5.1. Credencial de votante . . . 25
6.5.2. Boleta de sufragio . . . 26
6.5.3. Paquete de voto . . . 26

7. Propiedades de Seguridad del Protocolo 27
7.1. Modelo de seguridad . . . 27
7.2. Propiedad 1: Confidencialidad del voto . . . 27
7.3. Propiedad 2: Integridad del voto . . . 27
7.4. Propiedad 3: Autenticidad votante-autoridad . . . 28
7.5. Propiedad 4: Unicidad del voto . . . 28
7.6. Propiedad 5: Freshness y anti-replay . . . 28
7.7. Supuestos del modelo . . . 28

III Implementación y Evaluación Empírica 29

8. Implementación del Protocolo 31
8.1. Arquitectura por capas . . . 31
8.2. Motor criptográfico C — PQClean . . . 31
8.3. Wrapper Python (ctypes) . . . 32
8.4. Módulos Python del protocolo . . . 32
8.5. Validación KAT (Known Answer Tests) . . . 33

9. Benchmarking y Análisis Estadístico 35
9.1. Diseño experimental . . . 35
9.2. Latencia por voto (extremo a extremo) . . . 35
9.3. Apertura de ánfora — Escrutinio . . . 36
9.4. Micro-benchmarks por operación atómica . . . 36
9.5. Sobrecarga de comunicación . . . 36
9.6. Análisis estadístico inferencial . . . 37
9.6.1. Validación de la hipótesis . . . 37
9.6.2. Tamaño del efecto — Cohen's d . . . 37

IV Verificación Formal y Conclusiones **39**

10. Verificación Formal con Tamarin Prover **41**
10.1. ¿Para qué sirve la verificación formal? . . . 41
10.2. Abstracción criptográfica en el modelo . . . 41
10.3. Lemas verificados . . . 42

11. Suite de Pruebas y Calidad del Código **43**
11.1. Suite de pruebas automatizadas (pytest) . . . 43
11.2. Métricas de calidad . . . 43

12. Conclusiones y Trabajo Futuro **45**
12.1. Síntesis de resultados . . . 45
12.1.1. Respecto a la hipótesis . . . 45
12.1.2. Ventajas del protocolo PQC . . . 45
12.2. Limitaciones . . . 45
12.3. Trabajo futuro . . . 46

A. Notación Matemática **47**

B. Glosario de Términos **49**

C. Flujo del Protocolo en Notación Alice-Bob **51**

D. Comparación PQC vs. Clásico **53**

Referencias **55**

Índice de figuras

Índice de cuadros

1.1. Estándares PQC del NIST (2024) 4

2.1. Complejidad de los mejores algoritmos para SVP en rango n 7

3.1. Parámetros criptográficos de ML-KEM-768 (FIPS 203) 11

5.1. Parámetros de ML-DSA-65 (FIPS 204, nivel 3) 19
5.2. ML-DSA-65 vs. ECDSA P-256 19

6.1. Roles del protocolo de votación 23
6.2. Primitivas criptográficas del protocolo 24

8.1. Módulos Python del protocolo PQC 32
8.2. Resultados de validación KAT — NIST ACVP 33

9.1. Diseño del experimento 35
9.2. Latencia media por voto 35
9.3. Desencapsulamiento/descifrado por boleta en escrutinio 36
9.4. Tiempos por operación individual (100 repeticiones) 36
9.5. Comparación de tamaños de estructuras 36
9.6. Tamaño del efecto (Cohen's d) por escenario 37

10.1. Resultados de verificación formal con Tamarin Prover 42

11.1. Suite de pruebas automatizadas 43
11.2. Métricas de calidad del artefacto 43

A.1. Notación utilizada en el libro 47

D.1. Comparación completa: protocolo PQC vs. Clásico 53

Prefacio Pedagógico

Este libro ha sido concebido con un objetivo didáctico primordial: **democratizar la comprensión matemática y algorítmica de la criptografía basada en retículos (*Lattice-Based Cryptography*)**, en el marco del Proyecto de Grado de la Carrera de Informática de la Universidad Mayor de San Andrés (UMSA), titulado:

> *"Protocolo para la votación electrónica basado en criptografía post-cuántica de retículos".*

Durante décadas, la enseñanza de la criptografía asimétrica estuvo dominada exclusivamente por dos problemas de teoría de números clásica: la *dificultad de factorizar números enteros compuestos* (RSA) y el *problema del logaritmo discreto* sobre cuerpos finitos o curvas elípticas (Diffie-Hellman, ECDSA). Sin embargo, la computación cuántica, mediante el histórico algoritmo de Peter Shor (1994), ha demostrado que ambos pilares colapsarán irremediablemente en tiempo polinomial una vez que se construyan computadoras cuánticas a gran escala.

Ante esta realidad ineludible, el Instituto Nacional de Estándares y Tecnología de los Estados Unidos (NIST) concluyó en agosto de 2024 un proceso mundial de estandarización que duró ocho años, publicando formalmente:

- **FIPS 203**: Mecanismo de Encapsulamiento de Claves basado en Retículos de Módulo (ML-KEM, derivado de CRYSTALS-Kyber).
- **FIPS 204**: Algoritmo de Firmas Digitales basado en Retículos de Módulo (ML-DSA, derivado de CRYSTALS-Dilithium).

Ambos estándares descansan en un dominio matemático completamente distinto: **los retículos euclidianos y los problemas con error (*Learning With Errors — LWE*)**.

Cómo leer este libro

Este texto está diseñado para ser *autocontenido y progresivo*. No se asumen conocimientos avanzados de teoría de números algebraicos; basta con nociones elementales de álgebra lineal y aritmética modular.

El libro se organiza en cuatro partes interconectadas:

1. **Parte I: Fundamentos Matemáticos de los Retículos**: Se introduce la geometría de los retículos en $\mathbb{R}^n$, el paralelepípedo fundamental, el determinante, la ortogonalización de Gram-Schmidt, los problemas duros (SVP, CVP, SIS) y el concepto revolucionario de retículos q-arios.
2. **Parte II: De LWE a los Estándares NIST FIPS 203 y 204**: Se explica cómo añadir un pequeño ruido a un sistema de ecuaciones lineales destruye la eliminación gaussiana, cómo se transita a anillos ciclotómicos $R_q = \mathbb{Z}_q[x]/(x^n + 1)$, cómo opera la Transformada Teórica de Números (NTT) para multiplicar polinomios en $O(n \log n)$, y el funcionamiento detallado paso a paso de ML-KEM-768 y ML-DSA-65.

3. **Parte III: El Protocolo de Votación Electrónica PQC**: Se traslada toda la teoría matemática al problema real de la votación electrónica: entidades (Votante, Autoridad de Registro, Servidor de Votación, Autoridad de Conteo), ciclo de vida de la boleta, confidencialidad, integridad y no duplicidad.

4. **Parte IV: Evaluación Empírica, Benchmarking y Verificación**: Se exponen los resultados de 300 experimentos con validación estadística inferencial (Mann-Whitney U, Cohen's d), la verificación formal de lemas de seguridad en Tamarin Prover y el mapeo directo con el código fuente del repositorio.

Para maximizar la comprensión pedagógica, a lo largo del texto se incluyen cuatro tipos de recuadros interactivos:

- **Intuición Clave**: Explica la razón conceptual profunda detrás de una fórmula o diseño algorítmico.
- **Ejemplo Numérico Paso a Paso**: Cálculos con números pequeños (dimensiones $n = 2$ o $n = 4$, primos como $q = 17$) resueltos paso a paso para que el lector replique el proceso a mano.
- **Alerta Didáctica / Error Común**: Advierte sobre malentendidos frecuentes en la literatura.
- **Conexión con el Código del Proyecto**: Muestra exactamente qué línea y función de `src/pqc/` implementa dicho concepto matemático.

Facultad de Ciencias Puras y Naturales, UMSA — La Paz, 2026

Parte I

Fundamentos Matemáticos de los Retículos

Capítulo 1

El Desafío Cuántico y la Necesidad de Retículos

1.1. El ocaso de la criptografía de clave pública tradicional

Desde 1976, la seguridad de las comunicaciones en Internet ha dependido de dos familias de problemas computacionales:

1. **Factorización de números enteros compuestos**: Si $N = p \cdot q$, donde p y q son primos grandes secretos, recuperar p y q a partir de N requiere tiempo subexponencial con el algoritmo de criba del cuerpo de números de carácter general (GNFS):

$$\mathcal{O}\left(\exp\left(\sqrt[3]{\tfrac{64}{9}}(\ln N)^{1/3}(\ln\ln N)^{2/3}\right)\right).$$

   Para N de 2048 bits, este cálculo supera la capacidad combinada de todos los supercomputadores clásicos del planeta.

2. **Problema del Logaritmo Discreto (DLP)**: En un grupo cíclico $\mathbb{G}$ de orden p con generador g, dado $y = g^x \in \mathbb{G}$, calcular el exponente secreto $x = \log_g(y)$. Esta es la base de Diffie-Hellman y de las curvas elípticas (ECDSA).

1.1.1. El Algoritmo de Shor (1994)

En 1994, Peter Shor publicó un resultado revolucionario: en una computadora cuántica con registros de superposición coherente, tanto la factorización entera como el logaritmo discreto se reducen al **problema del hallazgo del período** de una función periódica $f(x) = a^x \pmod N$.

Mediante la **Transformada de Fourier Cuántica (QFT)**, una computadora cuántica puede encontrar el período r en tiempo polinomial:

$$\mathcal{O}\left((\log N)^2 \log(\log N) \log(\log(\log N))\right) \approx \mathcal{O}((\log N)^3).$$

Cuadro 1.1: Complejidad de resolución frente a computadoras clásicas y cuánticas

Problema / Algoritmo	**Mejor algoritmo clásico**	**Algoritmo de Shor (Cuántico)**
RSA-2048 (Factorización)	Subexponencial: $\sim 2^{112}$ ops	Polinomial: $\mathcal{O}((\log N)^3) \approx 10^9$ ops cuánticas
ECDSA P-256 (Log. Discreto)	Exponencial: $\sim 2^{128}$ ops	Polinomial: $\mathcal{O}(n^3) \approx 10^7$ ops cuánticas
Retículos (SVP / LWE)	**Exponencial:** $\sim 2^{0{,}292d}$	**Exponencial:** $\sim 2^{0{,}265d}$

► Por qué los retículos resisten a Shor

El algoritmo de Shor explota la estructura algebraica de los **grupos abelianos finitos**: encontrar un subgrupo o un período en una función homomórfica es una tarea para la cual la superposición cuántica es extraordinariamente eficaz. Los problemas en retículos euclidianos (como encontrar el vector más corto en una cuadrícula de dimensión 768) no poseen esta periodicidad unidimensional fácilmente separable, por lo que los algoritmos cuánticos conocidos solo ofrecen aceleraciones mínimas (como el algoritmo de Grover, que apenas reduce la búsqueda de 2^b a $2^{b/2}$).

1.2. La amenaza "Harvest Now, Decrypt Later" en sistemas electorales

Un error común es considerar que la amenaza cuántica es un problema del año 2035 o 2040. En aplicaciones comunes de navegación web efímera, un retardo en la migración podría ser tolerable. Sin embargo, en un **sistema de votación electrónica**:

1. **Secreto del voto a largo plazo**: La Constitución política y las leyes electorales exigen que la elección de un ciudadano permanezca secreta indefinidamente o durante décadas.
2. **Ataque de captura pasiva**: Un adversario estatal o una organización con grandes recursos puede interceptar y almacenar todas las boletas de voto cifradas transmitidas durante la elección actual (*Harvest Now*).
3. **Descifrado retrospectivo**: Dentro de 10 a 15 años, cuando se disponga de un procesador cuántico universal, el adversario ejecutará Shor sobre las claves públicas clásicas y revelará por quién votó cada ciudadano (*Decrypt Later*), quebrando retroactivamente la democracia.

Por este motivo imperativo, el diseño de un protocolo electoral robusto debe adoptar criptografía resistente a ataques cuánticos desde su origen.

1.3. El proceso de estandarización del NIST (2016–2024)

En 2016, el NIST convocó a la comunidad científica internacional a enviar candidatos post-cuánticos (PQC). Tras cuatro rigurosas rondas de análisis criptoanalítico, en agosto de 2024 se promulgaron los primeros estándares oficiales:

1. **FIPS 203 — ML-KEM** (*Module Lattice-based Key Encapsulation Mechanism*): Estándar principal para encapsulamiento y establecimiento de claves de sesión asimétricas.
2. **FIPS 204 — ML-DSA** (*Module Lattice-based Digital Signature Algorithm*): Estándar principal para firmas digitales y autenticación de integridad.
3. **FIPS 205 — SLH-DSA** (*Stateless Hash-based Digital Signature Algorithm*): Algoritmo de firma basado exclusivamente en funciones hash (respaldo seguro ante eventuales debilidades teóricas en retículos).

Este proyecto de grado implementa y evalúa los dos algoritmos de retículos de módulo más destacados: **ML-KEM-768** y **ML-DSA-65**, ambos clasificados en el **Nivel de Seguridad 3 del NIST** ($\approx$ 192 bits de seguridad clásica, equivalente a la resistencia de AES-192).

Capítulo 2

Geometría de los Retículos en Espacios Euclidianos

2.1. ¿Qué es un retículo matemático?

De manera intuitiva, un **retículo** (*lattice* en inglés) es una generalización multidimensional de una cuadrícula infinita regular y periódica de puntos en el espacio euclidiano $\mathbb{R}^m$.

Definición: Retículo (Lattice)

Sea $\{\mathbf{b}_1, \mathbf{b}_2, \ldots, \mathbf{b}_n\}$ un conjunto de n vectores linealmente independientes en el espacio euclidiano $\mathbb{R}^m$ (con $n \leq m$). El **retículo** $\mathcal{L}$ generado por esta base es el conjunto discreto formado por todas las combinaciones lineales enteras de dichos vectores:

$$\mathcal{L} = \mathcal{L}(\mathbf{b}_1, \ldots, \mathbf{b}_n) = \left\{ \sum_{i=1}^{n} z_i \mathbf{b}_i \;\middle|\; z_i \in \mathbb{Z} \right\}.$$

La matriz $\mathbf{B} = [\mathbf{b}_1 \mid \mathbf{b}_2 \mid \cdots \mid \mathbf{b}_n] \in \mathbb{R}^{m \times n}$, cuyas columnas son los vectores de la base, se denomina **matriz de base** del retículo. Se dice que n es la *dimensión* o *rango* del retículo, y m es la dimensión del espacio ambiente. Si $n = m$, el retículo se denomina de **rango completo**.

$\mathbb{Z}_q$ El retículo entero canónico $\mathbb{Z}^2$

En el plano $\mathbb{R}^2$, si tomamos la base ortonormal canónica:

$$\mathbf{b}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \mathbf{b}_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix},$$

el retículo generado es el conjunto de todos los puntos con coordenadas enteras:

$$\mathcal{L} = \mathbb{Z}^2 = \{(z_1, z_2)^T \mid z_1, z_2 \in \mathbb{Z}\}.$$

2.2. Bases equivalentes y matrices unimodulares

Una propiedad geométrica fundamental de los retículos —y el núcleo de su utilidad en criptografía asimétrica— es que **un mismo retículo puede ser generado por infinitas bases distintas**.

Definición: Matriz Unimodular

Una matriz cuadrada $\mathbf{U} \in \mathbb{Z}^{n\times n}$ con entradas enteras se denomina **unimodular** si su determinante es igual a ± 1:

$$\det(\mathbf{U}) \in \{+1, -1\}.$$

La consecuencia directa es que su matriz inversa $\mathbf{U}^{-1}$ también tiene coeficientes estrictamente enteros ($\mathbf{U}^{-1} \in \mathbb{Z}^{n\times n}$). El conjunto de matrices unimodulares $n \times n$ forma el grupo lineal general entero $\mathrm{GL}_n(\mathbb{Z})$.

Teorema: Equivalencia de Bases de un Retículo

Dos matrices de base $\mathbf{B}_1, \mathbf{B}_2 \in \mathbb{R}^{m\times n}$ generan exactamente el mismo retículo $\mathcal{L}(\mathbf{B}_1) = \mathcal{L}(\mathbf{B}_2)$ si y sólo si existe una matriz unimodular $\mathbf{U} \in \mathrm{GL}_n(\mathbb{Z})$ tal que:

$$\mathbf{B}_2 = \mathbf{B}_1 \cdot \mathbf{U}.$$

Demostración. Si $\mathbf{B}_2 = \mathbf{B}_1\mathbf{U}$ con $\mathbf{U} \in \mathbb{Z}^{n\times n}$, entonces cualquier combinación entera de columnas de $\mathbf{B}_2$ es $\mathbf{B}_2\mathbf{z} = \mathbf{B}_1(\mathbf{U}\mathbf{z})$. Como $\mathbf{U}\mathbf{z} \in \mathbb{Z}^n$, este vector pertenece a $\mathcal{L}(\mathbf{B}_1)$, luego $\mathcal{L}(\mathbf{B}_2) \subseteq \mathcal{L}(\mathbf{B}_1)$. Recíprocamente, dado que $\det(\mathbf{U}) = \pm 1$, existe $\mathbf{U}^{-1} \in \mathbb{Z}^{n\times n}$ tal que $\mathbf{B}_1 = \mathbf{B}_2\mathbf{U}^{-1}$, lo que prueba la inclusión contraria $\mathcal{L}(\mathbf{B}_1) \subseteq \mathcal{L}(\mathbf{B}_2)$. Por tanto, $\mathcal{L}(\mathbf{B}_1) = \mathcal{L}(\mathbf{B}_2)$. □

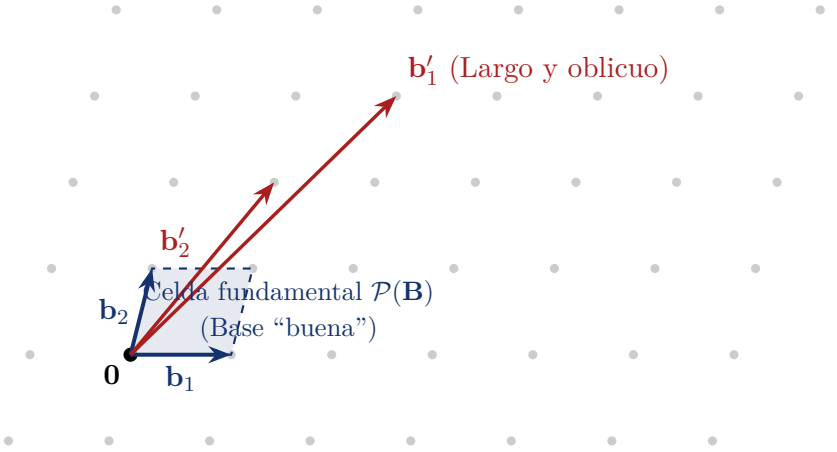


Figura 2.1: Representación de un retículo en $\mathbb{R}^2$: la base azul $\{\mathbf{b}_1, \mathbf{b}_2\}$ es corta y casi ortogonal (base "buena"), mientras que la base roja $\{\mathbf{b}_1', \mathbf{b}_2'\}$ está formada por vectores largos y casi colineales (base "mala"). Ambas generan exactamente el mismo conjunto de puntos.

► La asimetría criptográfica: Clave privada vs. Clave pública

La existencia de bases buenas y malas es la máquina de una sola vía fundamental:

- **Base buena (Clave privada)**: Vectores cortos y casi ortogonales. Permite decodificar fácilmente un punto con ruido y encontrar el punto del retículo más cercano.
- **Base mala (Clave pública)**: Se obtiene multiplicando la base buena por una matriz unimodular $\mathbf{U}$ muy dispersa. Sus vectores son larguísimos y casi paralelos. Cualquiera puede usarla para sumar vectores y cifrar un mensaje, pero resolver problemas geométricos con esta base es computacionalmente intratable.

2.3. El paralelepípedo fundamental y el determinante

Para medir la densidad geométrica de un retículo, se define la región del espacio cubierta por una celda unitaria.

Definición: Paralelepípedo Fundamental

Dada una matriz de base $\mathbf{B} = [\mathbf{b}_1, \ldots, \mathbf{b}_n] \in \mathbb{R}^{m \times n}$, su **paralelepípedo fundamental semiabierto** es el subconjunto de $\mathbb{R}^m$:

$$\mathcal{P}(\mathbf{B}) = \left\{ \sum_{i=1}^{n} x_i \mathbf{b}_i \;\middle|\; 0 \leq x_i < 1 \right\}.$$

El espacio euclidiano queda particionado de manera disjunta por las traslaciones del paralelepípedo fundamental sobre los puntos del retículo:

$$\mathbb{R}^n = \bigcup_{\mathbf{v} \in \mathcal{L}} (\mathbf{v} + \mathcal{P}(\mathbf{B})), \quad \text{donde } (\mathbf{v} + \mathcal{P}(\mathbf{B})) \cap (\mathbf{w} + \mathcal{P}(\mathbf{B})) = \emptyset \text{ si } \mathbf{v} \neq \mathbf{w}.$$

Definición: Determinante y Covolumen del Retículo

El **determinante** o covolumen de un retículo $\mathcal{L}(\mathbf{B})$ es el volumen n-dimensional de su paralelepípedo fundamental:

$$\det(\mathcal{L}) = \mathrm{vol}(\mathcal{P}(\mathbf{B})) = \sqrt{\det(\mathbf{B}^T \mathbf{B})}.$$

Si el retículo es de rango completo ($m = n$), la fórmula se simplifica a:

$$\det(\mathcal{L}) = |\det(\mathbf{B})|.$$

Proposición 2.1. *Invarianza del Determinante El determinante* $\det(\mathcal{L})$ *es un* ***invariante intrínseco*** *del retículo: no depende de la base elegida.*

Demostración. Si $\mathbf{B}_2 = \mathbf{B}_1 \mathbf{U}$ con $\det(\mathbf{U}) = \pm 1$, entonces:

$$|\det(\mathbf{B}_2)| = |\det(\mathbf{B}_1 \mathbf{U})| = |\det(\mathbf{B}_1)| \cdot |\det(\mathbf{U})| = |\det(\mathbf{B}_1)| \cdot 1 = |\det(\mathbf{B}_1)|.$$

□

2.4. Los mínimos sucesivos de Minkowski

¿Cuán corto puede ser un vector no nulo dentro de un retículo? Para responderlo formalmente, Hermann Minkowski introdujo los mínimos sucesivos.

Definición: Primer Mínimo $\lambda_1(\mathcal{L})$

El **primer mínimo** de un retículo $\mathcal{L}$, denotado $\lambda_1(\mathcal{L})$, es la longitud (norma euclidiana) del vector no nulo más corto de $\mathcal{L}$:

$$\lambda_1(\mathcal{L}) = \min_{\mathbf{v} \in \mathcal{L},\, \mathbf{v} \neq \mathbf{0}} \|\mathbf{v}\|.$$

De manera general, el i-ésimo mínimo $\lambda_i(\mathcal{L})$ es el radio más pequeño de una esfera centrada en el origen que contiene al menos i vectores linealmente independientes del retículo:

$$\lambda_i(\mathcal{L}) = \inf\{r > 0 \mid \dim(\mathrm{span}(\mathcal{L} \cap \mathcal{B}_n(0, r))) \geq i\}, \quad \text{con } \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n.$$

Teorema: Primer Teorema de Minkowski (1896)

Para cualquier retículo $\mathcal{L}$ de rango completo en $\mathbb{R}^n$, el vector más corto satisface la cota superior universal:

$$\lambda_1(\mathcal{L}) \leq \sqrt{n} \cdot (\det \mathcal{L})^{1/n}.$$

La constante de Hermite γ_n formaliza la razón entre $(\lambda_1)^2$ y $(\det \mathcal{L})^{2/n}$. Para dimensiones grandes ($n \geq 256$), como las utilizadas en criptografía, esta geometría determina la dificultad del problema SVP.

2.5. Ortogonalización de Gram-Schmidt y Reducción de Bases

Dada una base arbitraria $\mathbf{B} = [\mathbf{b}_1, \ldots, \mathbf{b}_n]$, el proceso clásico de **Gram-Schmidt (GSO)** genera una base ortogonal $\{\mathbf{b}_1^*, \ldots, \mathbf{b}_n^*\}$ en el espacio vectorial continuo $\mathbb{R}^n$:

$$\mathbf{b}_i^* = \mathbf{b}_i - \sum_{j=1}^{i-1} \mu_{i,j} \mathbf{b}_j^*, \quad \text{donde } \mu_{i,j} = \frac{\langle \mathbf{b}_i, \mathbf{b}_j^* \rangle}{\|\mathbf{b}_j^*\|^2}.$$

△ Gram-Schmidt no produce una base del retículo

Los coeficientes $\mu_{i,j}$ son casi siempre números fraccionarios o reales no enteros. Por tanto, los vectores ortogonales $\mathbf{b}_i^*$ **no pertenecen al retículo** $\mathcal{L}$ (excepto $\mathbf{b}_1^* = \mathbf{b}_1$). El desafío consiste en encontrar una base que esté en el retículo y sea *lo más cercana posible* a la ortogonal.

2.5.1. El Algoritmo LLL (Lenstra–Lenstra–Lovász, 1982)

Una base $\mathbf{B}$ se denomina **LLL-reducida** con parámetro $\delta \in (1/4, 1]$ (típicamente $\delta = 3/4$) si satisface:

1. **Condición de tamaño**: $|\mu_{i,j}| \leq 1/2$ para todo $1 \leq j < i \leq n$.
2. **Condición de Lovász**: $\delta \|\mathbf{b}_{i-1}^*\|^2 \leq \|\mathbf{b}_i^* + \mu_{i,i-1}\mathbf{b}_{i-1}^*\|^2$.

El algoritmo LLL corre en tiempo polinomial $\mathcal{O}(n^6 \log^3 C)$ y garantiza encontrar un vector no nulo tal que:

$$\|\mathbf{b}_1\| \leq 2^{(n-1)/4} \cdot \lambda_1(\mathcal{L}).$$

Aunque el factor de aproximación exponencial $2^{O(n)}$ es demasiado flojo para romper parámetros criptográficos con $n = 256$ (donde 2^{64} es inmenso), LLL y sus extensiones por bloques (**BKZ** — Block Korkine-Zolotarev) son el barómetro contra el cual se miden los parámetros de seguridad del NIST.

Capítulo 3

Problemas Computacionales Duros en Retículos

3.1. El problema del vector más corto (SVP)

Definición: SVP Exacto (Shortest Vector Problem)

Dado un retículo $\mathcal{L}$ representado por una base arbitraria $\mathbf{B} \in \mathbb{Z}^{m \times n}$, encontrar un vector no nulo $\mathbf{v} \in \mathcal{L} \setminus \{\mathbf{0}\}$ tal que:

$$\|\mathbf{v}\| = \lambda_1(\mathcal{L}).$$

El problema SVP exacto bajo la norma euclidiana fue demostrado ser NP-difícil bajo reducciones probabilísticas (Ajtai 1998, Micciancio 2001). En criptografía práctica se emplea la versión de aproximación:

Definición: SVP Aproximado (SVP_γ)

Dado un retículo $\mathcal{L}$ de dimensión n y un factor de aproximación $\gamma \geq 1$, encontrar un vector no nulo $\mathbf{v} \in \mathcal{L}$ que satisfaga:

$$\|\mathbf{v}\| \leq \gamma \cdot \lambda_1(\mathcal{L}).$$

Cuando $\gamma = \mathrm{poly}(n)$ es polinomial en la dimensión, ningún algoritmo clásico ni cuántico conocido puede resolver SVP_γ en tiempo subexponencial.

3.2. El problema del vector más cercano (CVP) y BDD

Definición: CVP (Closest Vector Problem)

Dado un retículo $\mathcal{L} \subset \mathbb{R}^m$ y un vector objetivo arbitrario $\mathbf{t} \in \mathbb{R}^m$ (que generalmente no está en el retículo), encontrar el punto del retículo $\mathbf{v} \in \mathcal{L}$ que minimiza la distancia euclidiana a $\mathbf{t}$:

$$\mathrm{dist}(\mathbf{t}, \mathcal{L}) = \min_{\mathbf{v} \in \mathcal{L}} \|\mathbf{t} - \mathbf{v}\|.$$

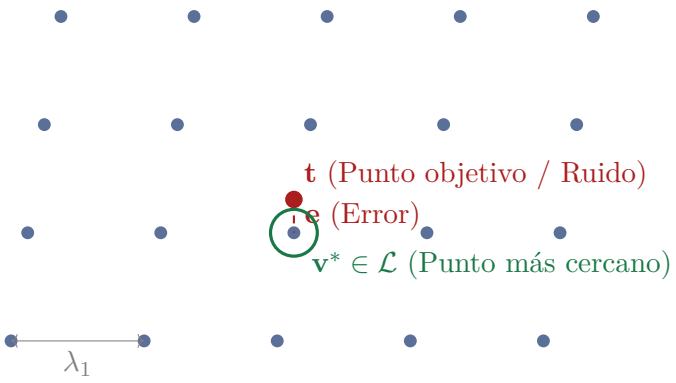


Figura 3.1: Geometría del problema CVP y BDD: un mensaje se codifica en un punto del retículo $\mathbf{v}^*$, perturbado por un pequeño vector de ruido $\mathbf{e}$ hasta llegar a $\mathbf{t} = \mathbf{v}^* + \mathbf{e}$. El descifrado consiste en resolver CVP para remover $\mathbf{e}$.

El problema de **Decodificación de Distancia Acotada (BDD)** es una variante restringida de CVP donde se promete que el objetivo $\mathbf{t}$ está a una distancia estrictamente menor que un radio garantizado:

$$\operatorname{dist}(\mathbf{t}, \mathcal{L}) < \frac{\lambda_1(\mathcal{L})}{2\gamma}.$$

En este caso, la solución es única. Esta es la operación matemática exacta realizada durante el **descifrado / desencapsulamiento en ML-KEM**.

3.2.1. El algoritmo de redondeo de Babai (1986)

Dada una base $\mathbf{B}$ y un punto objetivo $\mathbf{t}$, el algoritmo de Babai aproxima la solución expresando $\mathbf{t}$ en coordenadas de la base y redondeando cada coeficiente al entero más cercano:

$$\mathbf{v}_{\text{Babai}} = \mathbf{B} \cdot \left\lfloor \mathbf{B}^{-1}\mathbf{t} \right\rceil .$$

$\mathbb{Z}_q$ Fallo del redondeo de Babai con una base mala

Sea el retículo $\mathbb{Z}^2$ y tomemos dos bases distintas:

- Base buena (ortonormal): $\mathbf{B}_{\text{buena}} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$.
- Base mala: $\mathbf{B}_{\text{mala}} = \begin{pmatrix} 100 & 101 \\ 99 & 100 \end{pmatrix}$, con $\det(\mathbf{B}_{\text{mala}}) = 100 \cdot 100 - 101 \cdot 99 = 1$.

Tomemos el punto objetivo $\mathbf{t} = (0{,}45, 0{,}45)^T$. Con la base buena:

$$\mathbf{B}_{\text{buena}}^{-1}\mathbf{t} = \begin{pmatrix} 0{,}45 \\ 0{,}45 \end{pmatrix} \implies \left\lfloor \begin{pmatrix} 0{,}45 \\ 0{,}45 \end{pmatrix} \right\rceil = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \implies \mathbf{v} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

La distancia es $\|\mathbf{t} - \mathbf{v}\| = \sqrt{0{,}45^2 + 0{,}45^2} \approx 0{,}636$ (el punto más cercano correcto).

Con la base mala, $\mathbf{B}_{\text{mala}}^{-1} = \begin{pmatrix} 100 & -101 \\ -99 & 100 \end{pmatrix}$:

$$\mathbf{B}_{\text{mala}}^{-1}\mathbf{t} = \begin{pmatrix} 100(0{,}45) - 101(0{,}45) \\ -99(0{,}45) + 100(0{,}45) \end{pmatrix} = \begin{pmatrix} -0{,}45 \\ 0{,}45 \end{pmatrix}.$$

Redondeando al entero más próximo:

$$\left\lfloor \begin{pmatrix} -0{,}45 \\ 0{,}45 \end{pmatrix} \right\rceil = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \implies \mathbf{v} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

Sin embargo, si el objetivo es levemente modificado a $\mathbf{t}' = (0{,}55, 0{,}45)^T$:

$$\mathbf{B}_{\text{mala}}^{-1}\mathbf{t}' = \begin{pmatrix} 100(0{,}55) - 101(0{,}45) \\ -99(0{,}55) + 100(0{,}45) \end{pmatrix} = \begin{pmatrix} 55 - 45{,}45 \\ -54{,}45 + 45 \end{pmatrix} = \begin{pmatrix} 9{,}55 \\ -9{,}45 \end{pmatrix}.$$

Al redondear:

$$\left\lfloor \begin{pmatrix} 9{,}55 \\ -9{,}45 \end{pmatrix} \right\rceil = \begin{pmatrix} 10 \\ -9 \end{pmatrix} \implies \mathbf{v} = \begin{pmatrix} 100(10) + 101(-9) \\ 99(10) + 100(-9) \end{pmatrix} = \begin{pmatrix} 1000 - 909 \\ 990 - 900 \end{pmatrix} = \begin{pmatrix} 91 \\ 90 \end{pmatrix}.$$

¡El vector reticular hallado está a una distancia euclidiana de más de 127 unidades de $\mathbf{t}'$, a pesar de que el vector $(1, 0)^T$ estaba a distancia 0,64! Esto ilustra dramáticamente por qué una base mala no permite descifrar.

3.3. El problema de la solución entera corta (SIS)

Introducido por Miklós Ajtai en su célebre artículo de 1996, el problema SIS es la base primordial sobre la cual se erigen las **firmas digitales basadas en retículos** (como ML-DSA).

Definición: Problema SIS (Short Integer Solution)

Dada una matriz uniforme aleatoria $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ y una cota real positiva $\beta < q$, encontrar un vector entero no nulo $\mathbf{z} \in \mathbb{Z}^m \setminus \{\mathbf{0}\}$ tal que:

$$\mathbf{Az} \equiv \mathbf{0} \pmod{q} \quad \text{y} \quad \|\mathbf{z}\| \leq \beta.$$

Teorema: Reducción del peor caso al caso promedio de Ajtai (1996)

Si un algoritmo eficiente puede resolver $\text{SIS}_{q,\beta}$ sobre matrices aleatorias $\mathbf{A}$ distribuidas uniformemente en el **caso promedio**, entonces existe un algoritmo cuántico/clásico capaz de resolver el problema SVP_γ en **cualquier retículo arbitrario** en el **peor caso**.

Esta propiedad es excepcional: en criptografía RSA se asume que factorizar un N elegido al azar es difícil, pero podrían existir subclases de números fáciles. En retículos, generar una instancia aleatoria es matemáticamente tan difícil como la instancia más compleja que pueda existir en toda la geometría euclidiana.

Capítulo 4

Retículos q-arios y el Problema Learning With Errors (LWE)

4.1. Retículos q-arios: Conexión entre álgebra modular y geometría

Los criptosistemas modernos no trabajan con retículos continuos abstractos, sino con **retículos q-arios**: retículos enteros $\mathcal{L} \subseteq \mathbb{Z}^m$ que son periódicos respecto al hipercubo discreto $q\mathbb{Z}^m$, donde q es un número primo (o potencia de primo).

Definición: Retículos q-arios asociados a una matriz

Dada una matriz pública $\mathbf{A} \in \mathbb{Z}_q^{n\times m}$ (con $m > n$), se definen dos retículos duales en $\mathbb{Z}^m$:

$$\Lambda_q^{\perp}(\mathbf{A}) = \{\mathbf{z} \in \mathbb{Z}^m \mid \mathbf{A}\mathbf{z} \equiv \mathbf{0} \pmod q\}, \tag{4.1}$$

$$\Lambda_q(\mathbf{A}^T) = \{\mathbf{y} \in \mathbb{Z}^m \mid \mathbf{y} \equiv \mathbf{A}^T\mathbf{s} \pmod q \text{ para algún } \mathbf{s} \in \mathbb{Z}_q^n\}. \tag{4.2}$$

Nótese que:

- $\Lambda_q^{\perp}(\mathbf{A})$ es el núcleo (*kernel*) modular de la transformación lineal inducida por $\mathbf{A}$. Hallar un vector corto no nulo en este retículo es exactamente el problema **SIS**.
- $\Lambda_q(\mathbf{A}^T)$ es el espacio generado (*row space*) por las filas de $\mathbf{A}$. Hallar un vector cercano a un punto perturbado en este retículo es el problema **LWE**.

Proposición 4.1. *Determinante de los retículos q-arios Si las filas de $\mathbf{A} \in \mathbb{Z}_q^{n\times m}$ son linealmente independientes sobre $\mathbb{Z}_q$, entonces:*

$$\det(\Lambda_q^{\perp}(\mathbf{A})) = q^n \quad y \quad \det(\Lambda_q(\mathbf{A}^T)) = q^{m-n}.$$

4.2. El problema Learning With Errors (LWE)

Propuesto por Oded Regev en 2005, **Learning With Errors (LWE)** es la base matemática sobre la que se construye prácticamente toda la criptografía de retículos contemporánea.

Definición: Distribución LWE

Fijemos una dimensión de seguridad $n \geq 1$, un módulo primo $q \geq 2$, y una distribución de probabilidad de error discreto χ sobre $\mathbb{Z}$ concentrada fuertemente en valores pequeños (habitualmente una gaussiana discreta de desviación σ o una binomial centrada).

Para un vector secreto $\mathbf{s} \in \mathbb{Z}_q^n$, la **distribución LWE** $\mathcal{A}_{\mathbf{s},\chi}$ sobre $\mathbb{Z}_q^n \times \mathbb{Z}_q$ genera pares de la forma:

$$(\mathbf{a}, b) \in \mathbb{Z}_q^n \times \mathbb{Z}_q, \quad \text{donde } \mathbf{a} \xleftarrow{\$} \mathbb{Z}_q^n \text{ (uniforme)}, \; e \leftarrow \chi, \; b = \langle \mathbf{a}, \mathbf{s} \rangle + e \pmod{q}.$$

Existen dos formulaciones de este problema:

Definición: Search-LWE (Búsqueda)

Dadas m muestras independientes $(\mathbf{a}_i, b_i) \sim \mathcal{A}_{\mathbf{s},\chi}$, encontrar el vector secreto original $\mathbf{s} \in \mathbb{Z}_q^n$.

Definición: Decision-LWE (Decisión)

Distingir con ventaja no despreciable si m muestras $(\mathbf{a}_i, b_i)$ provienen de la distribución con estructura $\mathcal{A}_{\mathbf{s},\chi}$ o de la distribución uniforme $\mathcal{U}(\mathbb{Z}_q^n \times \mathbb{Z}_q)$.

4.3. Intuición didáctica: ¿Por qué el ruido rompe la eliminación gaussiana?

$\mathbb{Z}_q$ Sistema lineal exacto vs. Sistema LWE con ruido

Consideremos el cuerpo primo $\mathbb{Z}_{17}$ ($q = 17$) con dimensión secreta $n = 2$. Sea el secreto $\mathbf{s} = (3, 5)^T$.
1. Sin ruido ($e = 0$): Un atacante observa dos ecuaciones lineales:

$$2s_1 + 7s_2 \equiv 6 + 35 = 41 \equiv 7 \pmod{17},$$
$$11s_1 + 4s_2 \equiv 33 + 20 = 53 \equiv 2 \pmod{17}.$$

El atacante aplica eliminación de Gauss elemental: Multiplicando la primera ecuación por $11 \cdot 2^{-1} \equiv 11 \cdot 9 = 99 \equiv 14 \pmod{17}$:

$$14(2s_1 + 7s_2) \equiv 14(7) \implies 11s_1 + 13s_2 \equiv 13 \pmod{17}.$$

Restando de la segunda ecuación:

$$(4 - 13)s_2 \equiv 2 - 13 \implies -9s_2 \equiv -11 \implies 8s_2 \equiv 6 \pmod{17}.$$

Dado que $8^{-1} \equiv 15 \pmod{17}$, se tiene $s_2 \equiv 15 \cdot 6 = 90 \equiv 5 \pmod{17}$. Luego $2s_1 + 7(5) \equiv 7 \implies 2s_1 \equiv 7 - 1 = 6 \implies s_1 = 3$. **Resultado**: Se resolvió en fracciones de segundo con $\mathcal{O}(n^3)$ operaciones.
2. Con ruido pequeño ($e_1 = +1, e_2 = -1, e_3 = +1$): Las ecuaciones que recibe el adversario ahora son:

$$2s_1 + 7s_2 \equiv 7 + 1 = 8 \pmod{17},$$
$$11s_1 + 4s_2 \equiv 2 - 1 = 1 \pmod{17},$$
$$5s_1 + 12s_2 \equiv (15 + 60) + 1 = 75 + 1 \equiv 7 + 1 = 8 \pmod{17}.$$

Si el adversario intenta aplicar eliminación de Gauss sobre las dos primeras ecuaciones:

$$8s_2 \equiv 1 - 14(8) = 1 - 112 = 1 - 10 \equiv 8 \pmod{17} \implies s_2 \equiv 1 \pmod{17}.$$

Sustituyendo $s_2 = 1$: $2s_1 \equiv 8 - 7(1) = 1 \implies s_1 \equiv 9 \cdot 1 = 9 \pmod{17}$. El atacante obtiene la supuesta solución $\mathbf{s}' = (9, 1)^T$. Pero al probarla en la tercera ecuación:

$$5(9) + 12(1) = 45 + 12 = 57 \equiv 6 \not\equiv 8 \pmod{17}.$$

El sistema es totalmente inconsistente! El pequeño error $e \in \{-1, 0, 1\}$ se propaga multiplicado por los inversos modulares (como $2^{-1} = 9$), destruyendo la coherencia lineal y convirtiendo el problema en uno de optimización combinatoria no lineal cuya complejidad crece exponencialmente con la dimensión n.

Teorema: Dureza de LWE (Regev 2005)

Para cualquier módulo $q \leq \text{poly}(n)$ y parámetro de error $\alpha q \geq \sqrt{n}$, si existe un algoritmo eficiente que resuelve $\text{LWE}_{n,q,\chi}$, entonces existe un algoritmo cuántico eficiente para resolver $\text{GapSVP}_{\widetilde{\mathcal{O}}(n/\alpha)}$ y $\text{SIVP}_{\widetilde{\mathcal{O}}(n/\alpha)}$ en cualquier retículo euclidiano en el peor caso.

4.4. Distribuciones de error: De la Gaussiana a la CBD

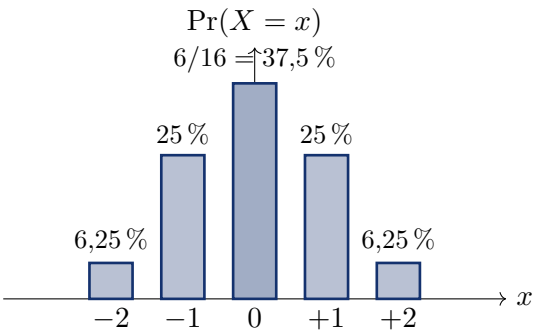
Teóricamente, LWE emplea una distribución gaussiana discreta $D_{\mathbb{Z},\sigma}$. No obstante, generar muestras gaussianas continuas de alta precisión en microprocesadores es sumamente vulnerable a **ataques de canal lateral** (*side-channel timing attacks*).

Por ello, en los estándares modernos del NIST (ML-KEM y ML-DSA) se adoptó la **Distribución Binomial Centrada** (CBD_η).

Definición: Distribución Binomial Centrada (CBD_η)

Para un entero $\eta \geq 1$, la distribución CBD_η sobre $\mathbb{Z}$ se obtiene muestreando 2η bits aleatorios independientes e idénticamente distribuidos $(a_1, \ldots, a_\eta, b_1, \ldots, b_\eta) \in \{0,1\}^{2\eta}$ y calculando:

$$x = \sum_{i=1}^{\eta} a_i - \sum_{i=1}^{\eta} b_i \in \{-\eta, -\eta+1, \ldots, 0, \ldots, \eta-1, \eta\}.$$



Distribución CBD_2 (usada en ML-KEM-768)

Figura 4.1: Forma acampanada discreta de la distribución CBD_2: su media es exactamente cero y su varianza es $\sigma^2 = \eta/2 = 1$. Se calcula en tiempo constante sumando bits generados por SHAKE-256.

Parte II

De LWE a los Estándares NIST FIPS 203 y 204

Capítulo 5

De LWE a Ring-LWE (RLWE): Polinomios y Retículos Ideales

5.1. El cuello de botella del LWE estándar

Aunque el problema LWE estándar es teóricamente impecable, adolece de una limitación práctica severa para sistemas embebidos y comunicaciones de alta velocidad: **el tamaño de la matriz pública A**.

Para una dimensión de seguridad de $n = 768$ y $m \approx 1024$ muestras con módulo $q \approx 4096$ (12 bits por elemento):

$$\text{Tamaño de } \mathbf{A} = n \times m \times \log_2(q) \approx 768 \times 1024 \times 12 \text{ bits} \approx 9{,}43 \text{ Megabits} \approx 1{,}18 \text{ Megabytes.}$$

Transmitir más de un megabyte por cada clave pública en una red de votación haría que el ancho de banda colapsara bajo concurrencia.

5.2. El anillo ciclotómico $R_q = \mathbb{Z}_q[x]/(x^n + 1)$

Para reducir el tamaño de las claves de $\mathcal{O}(n^2)$ a $\mathcal{O}(n)$, Lyubashevsky, Peikert y Regev (2010) introdujeron **Ring-LWE (RLWE)**, trasladando los vectores del álgebra lineal al **anillo de polinomios cociente**:

$$R_q = \mathbb{Z}_q[x]/(x^n + 1), \quad \text{donde } n \text{ es una potencia de 2 } (n = 2^d).$$

Definición: Elementos del anillo R_q

Un elemento $a(x) \in R_q$ es un polinomio de grado a lo sumo $n - 1$ con coeficientes en $\mathbb{Z}_q$:

$$a(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{n-1} x^{n-1}, \quad a_i \in \mathbb{Z}_q.$$

La adición se realiza sumando coeficientes módulo q. La multiplicación se realiza mediante la multiplicación usual de polinomios reducida por el polinomio mónico $x^n + 1$ y coeficientes módulo q:

$$c(x) = a(x) \cdot b(x) \pmod{x^n + 1} \pmod{q}.$$

5.3. La regla negacíclica y la correspondencia matricial

¿Qué significa geométricamente multiplicar por x módulo $(x^n + 1)$? Dado que $x^n \equiv -1 \pmod{x^n + 1}$, al multiplicar un monomio de grado máximo $a_{n-1}x^{n-1} \cdot x = a_{n-1}x^n \equiv -a_{n-1}$.

Esto produce una **rotación de coeficientes con cambio de signo (rotación negacíclica)**:

$$x \cdot (a_0 + a_1 x + \cdots + a_{n-1}x^{n-1}) \equiv -a_{n-1} + a_0 x + a_1 x^2 + \cdots + a_{n-2}x^{n-1}.$$

Teorema: Correspondencia Polinomio-Matriz

Multiplicar por un polinomio fijo $a(x) \in R_q$ sobre un polinomio $s(x) \in R_q$ es rigurosamente equivalente a multiplicar una **matriz circulante negacíclica** $\mathbf{M}_a \in \mathbb{Z}_q^{n\times n}$ por el vector de coeficientes $\mathbf{s} = (s_0, s_1, \ldots, s_{n-1})^T$:

$$\mathbf{M}_a = \begin{pmatrix} a_0 & -a_{n-1} & -a_{n-2} & \cdots & -a_1 \\ a_1 & a_0 & -a_{n-1} & \cdots & -a_2 \\ a_2 & a_1 & a_0 & \cdots & -a_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n-1} & a_{n-2} & a_{n-3} & \cdots & a_0 \end{pmatrix} \in \mathbb{Z}_q^{n\times n}.$$

► Ahorro drástico de memoria

Gracias a la estructura algebraica del anillo, **un solo polinomio $a(x)$ de n coeficientes representa y genera toda una matriz cuadrada de $n \times n$ entradas**. No es necesario almacenar ni transmitir las n^2 posiciones; sólo se transmiten los n coeficientes de la primera columna. Para $n = 256$, esto reduce el tamaño de la clave en un factor de 256.

$\mathbb{Z}_q$ Multiplicación polinomial módulo $x^4 + 1$ sobre $\mathbb{Z}_{17}$

Sean $n = 4, q = 17$, y los polinomios:

$$a(x) = 2 + 5x + 3x^2 + x^3, \quad s(x) = 1 + 2x + 0x^2 + 4x^3.$$

Calculamos el producto clásico $a(x) \cdot s(x)$:

$$\begin{aligned} a(x) \cdot s(x) &= 2(1 + 2x + 4x^3) + 5x(1 + 2x + 4x^3) + 3x^2(1 + 2x + 4x^3) + x^3(1 + 2x + 4x^3) \\ &= (2 + 4x + 8x^3) + (5x + 10x^2 + 20x^4) + (3x^2 + 6x^3 + 12x^5) + (x^3 + 2x^4 + 4x^6). \end{aligned}$$

Agrupando por potencias de x:

$$= 2 + (4 + 5)x + (10 + 3)x^2 + (8 + 6 + 1)x^3 + (20 + 2)x^4 + 12x^5 + 4x^6.$$

Aplicando la reducción $x^4 \equiv -1, x^5 \equiv -x, x^6 \equiv -x^2$:

$$\begin{aligned} &\equiv 2 + 9x + 13x^2 + 15x^3 + 22(-1) + 12(-x) + 4(-x^2) \\ &= (2 - 22) + (9 - 12)x + (13 - 4)x^2 + 15x^3 \\ &= -20 - 3x + 9x^2 + 15x^3 \pmod{17} \\ &\equiv 14 + 14x + 9x^2 + 15x^3. \end{aligned}$$

Ahora verifiquemos con la matriz circulante negacíclica $\mathbf{M}_a$:

$$\mathbf{M}_a\mathbf{s} = \begin{pmatrix} 2 & -1 & -3 & -5 \\ 5 & 2 & -1 & -3 \\ 3 & 5 & 2 & -1 \\ 1 & 3 & 5 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 0 \\ 4 \end{pmatrix} \equiv \begin{pmatrix} 2 & 16 & 14 & 12 \\ 5 & 2 & 16 & 14 \\ 3 & 5 & 2 & 16 \\ 1 & 3 & 5 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 0 \\ 4 \end{pmatrix} \pmod{17}.$$

Calculando fila por fila:

- Fila 0: $2(1) + 16(2) + 14(0) + 12(4) = 2 + 32 + 48 = 82 = 4 \times 17 + 14 \equiv 14$.
- Fila 1: $5(1) + 2(2) + 16(0) + 14(4) = 5 + 4 + 56 = 65 = 3 \times 17 + 14 \equiv 14$.
- Fila 2: $3(1) + 5(2) + 2(0) + 16(4) = 3 + 10 + 64 = 77 = 4 \times 17 + 9 \equiv 9$.
- Fila 3: $1(1) + 3(2) + 5(0) + 2(4) = 1 + 6 + 8 = 15 \equiv 15$.

El vector resultante es $(14, 14, 9, 15)^T$, ¡exactamente los coeficientes del polinomio calculado a mano!

Capítulo 6

La Transformada Teórica de Números (NTT)

6.1. La necesidad de acelerar el producto polinomial

En el capítulo anterior comprobamos que multiplicar dos polinomios de grado $n-1$ mediante el método tradicional de convolución requiere n^2 multiplicaciones escalares en $\mathbb{Z}_q$. Para $n = 256$, son $256^2 = 65,536$ operaciones.

Para lograr los tiempos de latencia inferiores a 2 milisegundos requeridos por nuestro protocolo de votación, se utiliza la **Transformada Teórica de Números (NTT)**, que es la adaptación exacta de la Transformada Rápida de Fourier (FFT) sobre cuerpos finitos discretos $\mathbb{Z}_q$. Con la NTT, la multiplicación se realiza en tiempo:

$$\mathcal{O}(n \log n) \approx 256 \times 8 = 2048 \text{ operaciones},$$

¡más de 30 veces más rápido que la convolución cuadrática!

6.2. Condiciones para la existencia de la NTT

Para que exista una transformada discreta de Fourier de orden N sobre $\mathbb{Z}_q$, el cuerpo finito $\mathbb{Z}_q$ debe contener una **raíz primitiva N-ésima de la unidad** $\omega \in \mathbb{Z}_q$ tal que:

$$\omega^N \equiv 1 \pmod{q} \quad \text{y} \quad \omega^k \not\equiv 1 \pmod{q} \text{ para todo } 1 \leq k < N.$$

Por el Pequeño Teorema de Fermat, esto ocurre si y sólo si N divide a $q-1$:

$$q \equiv 1 \pmod{N}.$$

Para multiplicar en el anillo negacíclico $R_q = \mathbb{Z}_q[x]/(x^n+1)$, se requiere calcular una transformada de orden $2n$, puesto que $x^{2n} \equiv 1 \pmod{x^n+1}$.

6.3. La singularidad matemática de Kyber / ML-KEM: La NTT Incompleta

Aquí encontramos un detalle técnico extraordinario de ML-KEM que distingue la investigación seria de la repetición superficial:

- En ML-KEM, los parámetros son $n = 256$ y $q = 3329$.
- Verifiquemos si $2n = 512$ divide a $q-1$:

$$q - 1 = 3329 - 1 = 3328 = 256 \times 13 = 2^8 \times 13.$$

- 256 divide a 3328, pero 512 **no divide a** 3328 (ya que $3328/512 = 6{,}5$).

► Por qué Kyber usa una NTT de 128 bloques cuadráticos

Dado que $512 \nmid (q-1)$, el polinomio $x^{256}+1$ **no se puede descomponer en 256 factores lineales de grado 1** en $\mathbb{Z}_q[x]$. En su lugar, se descompone en **128 factores cuadráticos irreducibles de grado 2**:

$$x^{256}+1 \equiv \prod_{i=0}^{127}(x^2 - \zeta^{2\cdot \mathrm{bitrev}_7(i)+1}) \pmod{3329},$$

donde $\zeta = 17$ es una raíz 256-ésima de la unidad ($17^{256} \equiv 1 \pmod{3329}$). Por tanto, la transformada NTT de Kyber convierte un polinomio de 256 coeficientes en **128 parejas de coeficientes** $\hat{f}_i = (f_{2i} + f_{2i+1}x)$. La multiplicación en el dominio NTT de Kyber no es una simple multiplicación escalar punto a punto, sino **128 multiplicaciones de polinomios lineales de grado 1** módulo $(x^2 - \zeta^{2k+1})$.

En contraste, en ML-DSA (FIPS 204), el módulo es $q = 8,380,417$. Aquí:

$$q - 1 = 8380416 = 2^{23} \cdot 1023.$$

Dado que $512 = 2^9$ divide holgadamente a $q-1$, en ML-DSA sí existen raíces 512-ésimas de la unidad y la NTT se descompone completamente en 256 factores lineales de grado 1 (multiplicación escalar punto a punto pura).

6.4. Algoritmo Mariposa (Cooley-Tukey Butterfly)

La operación atómica elemental de la NTT es la **mariposa de Cooley-Tukey**. En cada etapa, toma un par de coeficientes (u, v) y los actualiza multiplicando por una potencia de la raíz (denominada *factor twiddle* ζ):

$$u' = u + \zeta \cdot v \pmod{q}, \tag{6.1}$$
$$v' = u - \zeta \cdot v \pmod{q}. \tag{6.2}$$

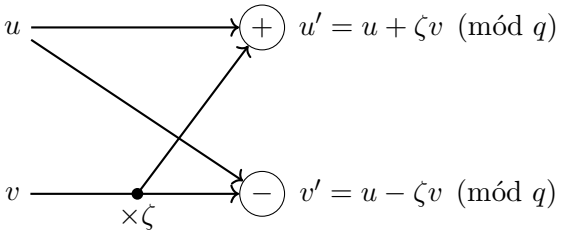


Figura 6.1: Diagrama de flujo de la operación elemental Mariposa (Butterfly) de Cooley-Tukey empleada en la implementación C de ML-KEM en `src/pqc/pqclean/`.

Capítulo 7

Module-LWE (MLWE): El Núcleo de Kyber y Dilithium

7.1. El equilibrio perfecto: ¿Por qué Módulos?

Hasta ahora hemos estudiado dos extremos:

- **LWE Estándar**: Máxima confianza de seguridad (sin estructura algebraica), pero claves enormes de cientos de kilobytes.
- **Ring-LWE (RLWE)**: Claves sumamente compactas, pero una estructura de anillo ideal muy rígida que podría ser vulnerable a ataques algebraicos especializados sobre ideales principales. Además, para aumentar la seguridad sólo se puede duplicar el grado n ($n = 256 \to 512 \to 1024$), lo que genera saltos bruscos de rendimiento.

La solución magistral adoptada por el NIST es **Module-LWE (MLWE)**, formalizada por Langlois y Stehlé en 2012.

En MLWE, se fija el grado polinomial en un valor constante y eficiente ($n = 256$) y se trabaja sobre **módulos libres de rango k sobre el anillo R_q**:

$$R_q^k = \underbrace{R_q \times R_q \times \cdots \times R_q}_{k \text{ copias de } R_q}.$$

Definición: Problema Module-LWE

Sea $n = 256$, $R_q = \mathbb{Z}_q[x]/(x^n + 1)$, y $k \geq 1$ la dimensión del módulo. Una instancia MLWE está definida por una matriz pública de polinomios:

$$\mathbf{A} \in R_q^{k \times k},$$

y dos vectores de polinomios con coeficientes pequeños muestreados de la distribución de error χ:

$$\mathbf{s} \in R_q^k \quad \text{(secreto)}, \qquad \mathbf{e} \in R_q^k \quad \text{(error)}.$$

El vector público generado es:

$$\mathbf{b} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e} \in R_q^k.$$

▶ Cómo se modula el nivel de seguridad

La ventaja arquitectónica de MLWE es que para cambiar de nivel de seguridad **no se modifica el código aritmético de la NTT ni el grado** $n = 256$; simplemente se incrementa el rango k de la matriz:

- Nivel 1 (AES-128 equiv.): $k = 2 \implies$ Dimensión del retículo $d = 2 \times 256 = 512$ (ML-KEM-512).
- **Nivel 3 (AES-192 equiv., usado en la tesis)**: $k = 3 \implies$ Dimensión $d = 3 \times 256 = 768$ (**ML-KEM-768**).
- Nivel 5 (AES-256 equiv.): $k = 4 \implies$ Dimensión $d = 4 \times 256 = 1024$ (ML-KEM-1024).

Capítulo 8

ML-KEM-768 (FIPS 203) en Profundidad Didáctica

8.1. Estructura general de un KEM

Un Mecanismo de Encapsulamiento de Claves (**KEM**) asimétrico no cifra mensajes arbitrariamente largos de forma directa. En su lugar, establece un **secreto simétrico compartido efímero de 256 bits ($K \in \{0,1\}^{32}$)** entre dos partes que se comunican por un canal público inseguro.

Consta de tres algoritmos:

1. $\mathsf{KeyGen}() \to (\mathtt{ek}, \mathtt{dk})$: Genera la clave pública de encapsulamiento $\mathtt{ek}$ y la clave privada de desencapsulamiento $\mathtt{dk}$.

2. $\mathsf{Encaps}(\mathtt{ek}) \to (K, \mathtt{ct})$: Usando la clave pública, genera el secreto compartido K y el texto cifrado (*ciphertext*) $\mathtt{ct}$.

3. $\mathsf{Decaps}(\mathtt{dk}, \mathtt{ct}) \to K$: Usando la clave privada y el texto cifrado, recupera con exactitud el mismo secreto K.

8.2. Parámetros oficiales de ML-KEM-768 (FIPS 203)

Cuadro 8.1: Especificación de parámetros de ML-KEM-768

Parámetro	**Valor**	**Significado matemático**
n	256	Grado del polinomio ciclotómico $x^n + 1$
q	3329	Módulo primo ($3329 = 13 \times 256 + 1$)
k	3	Rango del módulo (dimensión efectiva: $256 \times 3 = 768$)
η_1	2	Parámetro de ruido CBD_{η_1} para generación de claves $(\mathbf{s}, \mathbf{e})$
η_2	2	Parámetro de ruido CBD_{η_2} para encapsulamiento $(\mathbf{r}, \mathbf{e}_1, e_2)$
d_u	10	Bits de compresión para el vector $\mathbf{u}$ en el ciphertext
d_v	4	Bits de compresión para el escalar v en el ciphertext
Tamaños exactos en bytes		
Clave pública ($\mathtt{ek}$)	1 184 bytes	3 polinomios en NTT (3×384) + semilla de 32 bytes
Clave privada ($\mathtt{dk}$)	2 400 bytes	$\mathbf{s}$ (1152) + $\mathtt{ek}$ (1184) + $H(\mathtt{ek})$ (32) + semilla z (32)
Texto cifrado ($\mathtt{ct}$)	1 088 bytes	$3 \times 256 \times 10/8 = 960$ bytes ($\mathbf{u}$) $+256 \times 4/8 = 128$ bytes (v)
Secreto compartido (K)	32 bytes	256 bits criptográficamente fuertes para AES-256-GCM

8.3. Codificación de mensajes binarios en coeficientes polinomiales

Un mensaje efímero $m \in \{0,1\}^{32}$ contiene exactamente $32 \times 8 = 256$ bits. Cada bit $m_i \in \{0,1\}$ se asocia de forma biunívoca con el i-ésimo coeficiente del polinomio mensaje $m(x) \in R_q$:

$$m(x) = \sum_{i=0}^{255} m_i \cdot \left\lfloor \frac{q}{2} \right\rceil x^i = \sum_{i=0}^{255} m_i \cdot 1665\, x^i \pmod{3329}.$$

- Si $m_i = 0$, el coeficiente es 0.
- Si $m_i = 1$, el coeficiente es $\lfloor 3329/2 \rceil = 1665$.

Los valores 0 y 1665 están a la máxima distancia posible dentro del círculo modular $\mathbb{Z}_{3329}$, maximizando la tolerancia al ruido.

8.4. El cifrador de clave pública subyacente: Kyber.CPAPKE

8.4.1. Generación de claves (KeyGen)

1. Se genera una semilla aleatoria $\rho \in \{0,1\}^{32}$ que expande la matriz pública $\mathbf{A} \in R_q^{3\times 3}$ en el dominio NTT mediante SHAKE-128.
2. Se muestrean el secreto $\mathbf{s} \leftarrow \mathrm{CBD}_2^3$ y el error $\mathbf{e} \leftarrow \mathrm{CBD}_2^3$.
3. Se calcula en el dominio NTT:
$$\mathbf{t} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e} \in R_q^3.$$
4. Clave pública: $\mathtt{pk} = (\mathbf{t}, \rho)$. Clave privada: $\mathtt{sk} = \mathbf{s}$.

8.4.2. Cifrado (Encaps)

Para cifrar el mensaje $m(x)$ usando $\mathtt{pk} = (\mathbf{t}, \rho)$:

1. El emisor regenera la matriz $\mathbf{A}$ desde la semilla ρ.
2. Muestrea tres variables de ruido efímeras: un vector secreto $\mathbf{r} \leftarrow \mathrm{CBD}_2^3$, un vector de error $\mathbf{e}_1 \leftarrow \mathrm{CBD}_2^3$ y un error escalar $e_2 \leftarrow \mathrm{CBD}_2$.
3. Calcula las dos componentes del texto cifrado:
$$\mathbf{u} = \mathbf{A}^T\mathbf{r} + \mathbf{e}_1 \pmod{q} \in R_q^3, \tag{8.1}$$
$$v = \mathbf{t}^T\mathbf{r} + e_2 + \left\lfloor \frac{q}{2} \right\rceil m(x) \pmod{q} \in R_q. \tag{8.2}$$
4. El texto cifrado es $\mathtt{ct} = (\mathbf{u}, v)$.

8.4.3. Descifrado (Decaps)

El receptor, poseyendo el secreto $\mathbf{s}$, calcula:

$$m'(x) = v - \mathbf{s}^T\mathbf{u} \pmod{q}.$$

8.5. Demostración matemática de la cancelación de ruido

¿Por qué el receptor recupera exactamente el mensaje original $m(x)$? Desarrollemos algebraicamente la operación de descifrado:

$$\begin{aligned} v - \mathbf{s}^T\mathbf{u} &= \left(\mathbf{t}^T\mathbf{r} + e_2 + \left\lfloor \frac{q}{2} \right\rceil m\right) - \mathbf{s}^T\left(\mathbf{A}^T\mathbf{r} + \mathbf{e}_1\right) \\ &= \left((\mathbf{A}\mathbf{s} + \mathbf{e})^T\mathbf{r} + e_2 + \left\lfloor \frac{q}{2} \right\rceil m\right) - \mathbf{s}^T\mathbf{A}^T\mathbf{r} - \mathbf{s}^T\mathbf{e}_1 \\ &= \left(\mathbf{s}^T\mathbf{A}^T\mathbf{r} + \mathbf{e}^T\mathbf{r} + e_2 + \left\lfloor \frac{q}{2} \right\rceil m\right) - \mathbf{s}^T\mathbf{A}^T\mathbf{r} - \mathbf{s}^T\mathbf{e}_1. \end{aligned}$$

Nótese que los términos masivos que contienen a la matriz pública $\mathbf{s}^T\mathbf{A}^T\mathbf{r}$ **se cancelan exactamente entre sí**:

$$v - \mathbf{s}^T\mathbf{u} = \left\lfloor \frac{q}{2} \right\rceil m + \underbrace{(\mathbf{e}^T\mathbf{r} + e_2 - \mathbf{s}^T\mathbf{e}_1)}_{E(x)\text{ (Ruido total acumulado)}} \quad (\text{mód } q).$$

Teorema: Condición de Corrección Estricta

Para cada coeficiente $i \in \{0, \ldots, 255\}$, si el valor absoluto del ruido acumulado es estrictamente menor que un cuarto del módulo:

$$|E_i| < \frac{q}{4} = \frac{3329}{4} \approx 832{,}25,$$

la función de redondeo:

$$m_i' = \begin{cases} 0 & \text{si } |(v - \mathbf{s}^T\mathbf{u})_i| < \frac{q}{4} \text{ o } |(v - \mathbf{s}^T\mathbf{u})_i| > \frac{3q}{4}, \\ 1 & \text{si } \frac{q}{4} \leq (v - \mathbf{s}^T\mathbf{u})_i \leq \frac{3q}{4}, \end{cases}$$

recupera $m_i' = m_i$ sin un solo bit de error.

Dado que los coeficientes de $\mathbf{s}, \mathbf{e}, \mathbf{r}, \mathbf{e}_1$ provienen de CBD_2 y están acotados por ± 2, la probabilidad de que el ruido acumulado supere 832 es astronómicamente pequeña:

$$\Pr[\text{Fallo de decodificación en ML-KEM-768}] < 2^{-164}.$$

Esto es infinitamente menor que la probabilidad de que un rayo cósmico altere un bit en la memoria RAM del servidor.

8.6. La Transformación Fujisaki-Okamoto (FO): De CPA a IND-CCA2

El cifrador Kyber.CPAPKE sólo es seguro frente a ataques con texto plano escogido (CPA). Si un adversario pudiera enviar textos cifrados manipulados y observar si el descifrado falla, podría deducir la clave privada $\mathbf{s}$ mediante ataques de reacción.

Para alcanzar la seguridad máxima frente a ataques de texto cifrado adaptativo (**IND-CCA2**), FIPS 203 aplica la **Transformación de Fujisaki-Okamoto (FO) con Rechazo Implícito**:

1. Durante el encapsulamiento, las variables de ruido $\mathbf{r}, \mathbf{e}_1, e_2$ **no se eligen al azar**, sino que se derivan determinísticamente evaluando una función hash criptográfica (SHAKE-256) sobre el mensaje efímero m y la clave pública:

$$(\bar{K}, r) \leftarrow G(m \,\|\, H(\texttt{ek})).$$

2. Durante el desencapsulamiento, tras recuperar m', el receptor **vuelve a ejecutar el cifrado completo** usando las monedas $r' = G(m' \,\|\, H(\texttt{ek}))$, generando un texto cifrado de prueba $\texttt{ct}'$.

3. Si $\texttt{ct}' \neq \texttt{ct}$, significa que el texto cifrado recibido fue manipulado maliciosamente por un adversario. En lugar de emitir un mensaje de error que filtraría información de temporización, el algoritmo aplica **rechazo implícito**: devuelve una clave pseudoaleatoria $K = \mathrm{PRF}(z, \texttt{ct})$ computada a partir de un valor secreto $z \in \texttt{dk}$, indistinguible de ruido blanco para el atacante.

Capítulo 9

ML-DSA-65 (FIPS 204) en Profundidad Didáctica

9.1. El paradigma de firmas con retículos: ¿Por qué no un RSA inverso?

En RSA, firmar un documento consiste en la misma operación matemática que descifrar: computar $s = m^d \pmod{N}$.

En retículos, **esto es imposible**:

- El descifrado remueve un error pequeño e.
- Si firmar consistiera en resolver CVP para encontrar un punto del retículo cercano al hash del mensaje $H(m)$, las firmas sucesivas irían revelando la forma del paralelepípedo fundamental de la clave privada (ataque de Nguyen-Regev a GGH/NTRUSign).

Por ello, el estándar ML-DSA (FIPS 204), derivado del esquema CRYSTALS-Dilithium (Ducas et al., 2018), emplea el enfoque de **Protocolos de Identificación con Abortos de Lyubashevsky** transformados mediante la heurística de **Fiat-Shamir**.

9.2. ¿Por qué se necesitan abortos? La fuga geométrica

Consideremos el esquema de identificación elemental:

1. El firmante elige una máscara aleatoria $\mathbf{y}$ y publica el compromiso $\mathbf{w} = \mathbf{A}\mathbf{y} \pmod{q}$.
2. El verificador envía un desafío hash $c = H(m \,\|\, \mathbf{w})$.
3. El firmante responde con el vector:

$$\mathbf{z} = \mathbf{y} + c \cdot \mathbf{s}.$$

△ Fuga del secreto en firmas ingenuas

La firma $\mathbf{z} = \mathbf{y} + c\mathbf{s}$ depende directamente de la clave secreta $\mathbf{s}$. Aunque $\mathbf{y}$ sea uniforme dentro de una caja $[-\gamma_1, \gamma_1]^\ell$, la distribución de $\mathbf{z}$ es la convolución de $\mathbf{y}$ desplazada por $c\mathbf{s}$. Un atacante que recopile unos pocos cientos de firmas podría promediar los valores de $\mathbf{z}$ o analizar la densidad de sus bordes geométricos para reconstruir con precisión exacta la clave secreta $\mathbf{s}$!

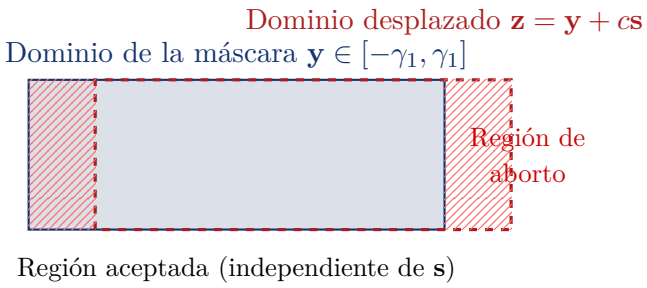


Figura 9.1: Mecanismo de Muestreo de Rechazo (*Rejection Sampling*) en ML-DSA: si la respuesta $\mathbf{z}$ cae cerca de los bordes del dominio (regiones sombreadas), se aborta la operación y se repite el proceso con una nueva máscara $\mathbf{y}$. Esto garantiza que las firmas válidas publicadas tengan una distribución geométrica perfectamente independiente del secreto $\mathbf{s}$.

9.3. Algoritmos de ML-DSA-65 (FIPS 204)

9.3.1. Parámetros de ML-DSA-65 (Nivel de Seguridad 3)

- Dimensiones de la matriz: $k = 6, \ell = 5$ sobre $R_q = \mathbb{Z}_q[x]/(x^{256} + 1)$.
- Módulo primo: $q = 8,380,417 = 2^{23} - 2^{13} + 1$.
- Cota de ruido secreto: $\eta = 4$.
- Rango de la máscara: $\gamma_1 = 2^{19} = 524,288$.
- Radio de redondeo: $\gamma_2 = (q-1)/32 = 261,888$.
- Umbral de aborto: $\beta = 120$.
- Tamaños: Clave pública = **1,952 bytes**, Clave privada = **4,032 bytes**, Firma = **3,309 bytes**.

9.3.2. Generación de Claves (KeyGen)

1. Expande la matriz $\mathbf{A} \in R_q^{6\times 5}$ a partir de una semilla pública ρ de 32 bytes.
2. Muestrea los vectores secretos $\mathbf{s}_1 \leftarrow S_\eta^5$ y $\mathbf{s}_2 \leftarrow S_\eta^6$ con coeficientes en $[-\eta, \eta]$.
3. Calcula el vector público $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2 \in R_q^6$.
4. Para comprimir la clave pública, descompone $\mathbf{t}$ en potencias de 2 mediante Power2Round:

$$\mathbf{t} = \mathbf{t}_1 \cdot 2^d + \mathbf{t}_0.$$

5. La clave pública contiene sólo la semilla ρ y los bits altos $\mathbf{t}_1$ (1,952 bytes).

9.3.3. Firma (Sign)

Algoritmo 1: ML-DSA.Sign($\mathtt{sk}, m$)

Salida: Firma digital $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$

$\kappa \leftarrow 0$; $(\rho, K, \mathrm{tr}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0) \leftarrow \mathtt{sk}$;
$\mathbf{A} \leftarrow \mathrm{ExpandA}(\rho)$;
$\mu \leftarrow H(\mathrm{tr} \,\|\, m)$;
repeat
  $\mathbf{y} \leftarrow \mathrm{ExpandMask}(K, \mu, \kappa)$; $\kappa \leftarrow \kappa + \ell$;
  $\mathbf{w} \leftarrow \mathbf{A}\mathbf{y}$;
  $\mathbf{w}_1 \leftarrow \mathrm{HighBits}(\mathbf{w}, 2\gamma_2)$;
  $\tilde{c} \leftarrow H(\mu \,\|\, \mathbf{w}_1)$; $c \leftarrow \mathrm{SampleInBall}(\tilde{c})$;
  $\mathbf{z} \leftarrow \mathbf{y} + c\mathbf{s}_1$;
  $\mathbf{r}_0 \leftarrow \mathrm{LowBits}(\mathbf{w} - c\mathbf{s}_2, 2\gamma_2)$;
  if $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$ ***o*** $\|\mathbf{r}_0\|_\infty \geq \gamma_2 - \beta$ **then**
    abortar y reintentar; `// Muestreo de rechazo`
  $\mathbf{h} \leftarrow \mathrm{MakeHint}(-c\mathbf{t}_0,\ \mathbf{w} - c\mathbf{s}_2 + c\mathbf{t}_0, 2\gamma_2)$;
  if $\|c\mathbf{t}_0\|_\infty \geq \gamma_2$ ***o*** *número de unos en* $\mathbf{h} > \omega$ **then**
    abortar y reintentar;
until *no abortar*;
return $\sigma \leftarrow (\tilde{c}, \mathbf{z}, \mathbf{h})$

9.3.4. Verificación (Verify)

El verificador recupera la matriz $\mathbf{A}$ desde la clave pública, verifica que $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$, y reconstruye los bits altos del compromiso usando el vector de pistas $\mathbf{h}$:

$$\mathbf{w}'_1 = \mathrm{UseHint}(\mathbf{h},\ \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d).$$

Calcula $c' = H(\mu \,\|\, \mathbf{w}'_1)$ y acepta la firma si y sólo si $c' = \tilde{c}$.

Parte III

El Protocolo de Votación Electrónica Post-Cuántica

Capítulo 10

Arquitectura y Diseño Formal del Protocolo

10.1. Objetivos del protocolo electoral

Un sistema de votación electrónica es una de las construcciones criptográficas más complejas que existen, debido a la tensión fundamental entre dos requisitos aparentemente contradictorios:

1. **Privacidad y secreto del voto**: Nadie, ni siquiera los administradores del sistema electoral o atacantes con recursos infinitos, debe poder asociar una identidad con el voto emitido.

2. **Integridad, autenticidad y unicidad**: La autoridad debe garantizar con certeza matemática que sólo los votantes debidamente registrados votaron, que cada votante habilitado sufragó exactamente una vez (no duplicidad), y que ningún voto fue alterado en tránsito o en la base de datos.

10.2. Entidades y separación criptográfica de poderes

Para evitar la existencia de un único punto de compromiso, el protocolo divide las responsabilidades entre cuatro roles estrictamente independientes:

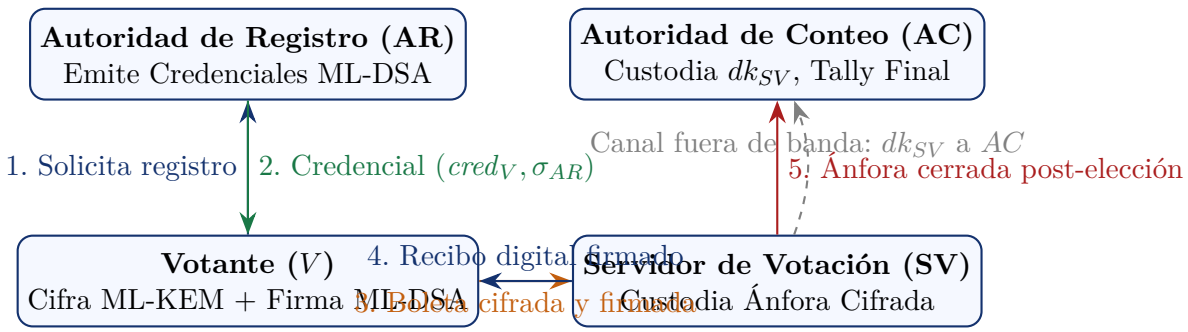


Figura 10.1: Diagrama de roles y flujos de comunicación del protocolo electoral post-cuántico.

Cuadro 10.1: Matriz de responsabilidades y claves del sistema

Rol	Entidad	Pares de claves PQC	Responsabilidad estricta
AR	Autoridad de Registro	ML-DSA-65 (sk_{AR}, pk_{AR})	Valida identidad ciudadana, firma credenciales.
AC	Autoridad de Conteo	Custodia dk_{SV}	Desencapsula boletas en el escrutinio final.
SV	Servidor de Votación	ML-KEM-768 (ek_{SV}, dk_{SV})	Valida credencial y firma, previene doble voto.
		ML-DSA-65 (sk_{SV}, pk_{SV})	Emite recibos firmados de depósito en ánfora.
V	Votante	ML-DSA-65 (sk_V, pk_V)	Cifra su voto, firma su boleta de forma efímera.

La Regla de Oro del Secreto Electoral

El Servidor de Votación (SV) tiene la lista de identidades ID_V y sus credenciales, pero **no posee la clave privada dk_{SV} para descifrar las boletas**. La Autoridad de Conteo (AC) posee la clave privada dk_{SV} para descifrar, pero **sólo recibe boletas anónimas del ánfora una vez concluida la votación**, sin poder correlacionar quién emitió cuál voto. Ninguna entidad por separado puede violar la privacidad del ciudadano.

10.3. Las 6 Fases Secuenciales del Protocolo

10.3.1. Fase 1: Inicialización y Configuración del Sistema

1. La AR genera su par de claves de firma: $(sk_{AR}, pk_{AR}) \leftarrow \mathsf{ML\text{-}DSA.KeyGen}()$.
2. El SV genera el par de claves de la elección: $(ek_{SV}, dk_{SV}) \leftarrow \mathsf{ML\text{-}KEM.KeyGen}()$.
3. El SV genera su par para emisión de recibos: $(sk_{SV}, pk_{SV}) \leftarrow \mathsf{ML\text{-}DSA.KeyGen}()$.
4. Mediante un canal seguro fuera de banda (tarjeta inteligente o ceremonia de claves), dk_{SV} se transfiere exclusivamente a la AC.
5. Se publican las claves públicas $pk_{AR}, ek_{SV}, pk_{SV}$ en el tablón electoral auditado.

10.3.2. Fase 2: Registro del Votante y Emisión de Credencial

1. El ciudadano V se presenta ante la AR y demuestra su identidad legal (ID_V).
2. La AR consulta el padrón electoral oficial. Si está habilitado:
3. Se genera un par de claves efímero para el votante: $(sk_V, pk_V) \leftarrow \mathsf{ML\text{-}DSA.KeyGen}()$.
4. La AR construye el cuerpo de la credencial:

$$cred_V = (ID_V \,\|\, pk_V \,\|\, \text{timestamp}).$$

5. La AR firma la credencial: $\sigma_{AR} = \mathsf{ML\text{-}DSA.Sign}(sk_{AR}, cred_V)$.
6. Se entrega al votante $(sk_V, cred_V, \sigma_{AR})$. El votante queda en estado "Registrado".

10.3.3. Fase 3: Emisión y Cifrado del Voto

El votante ingresa a la cabina de votación con su dispositivo seguro:

1. Selecciona su opción de sufragio $m \in \{\text{Candidato A}, \text{Candidato B}, \ldots\}$.
2. Ejecuta el encapsulamiento post-cuántico contra la clave pública de la elección:

$$(K, ct) \leftarrow \mathsf{ML\text{-}KEM.Encaps}(ek_{SV}), \quad \text{donde } ct \in \{0,1\}^{1088},\ K \in \{0,1\}^{32}.$$

3. Deriva una clave simétrica $K_{\text{AES}} = \text{HKDF-SHA3}(K)$ y un nonce aleatorio de 12 bytes.

4. Cifra el voto mediante cifrado autenticado simétrico:

$$c = \text{AES-256-GCM.Enc}(K_{\text{AES}}, m, \mathit{nonce}).$$

5. Construye el cuerpo de la boleta:

$$\mathit{boleta} = (\mathit{ct} \,\|\, c \,\|\, \mathit{nonce} \,\|\, \mathit{cred}_V \,\|\, \sigma_{AR} \,\|\, \text{timestamp}).$$

6. Firma digitalmente la boleta completa usando su clave privada efímera:

$$\sigma_V = \mathsf{ML\text{-}DSA.Sign}(\mathit{sk}_V, \mathit{boleta}).$$

7. El paquete final transmitido es $\mathcal{P} = (\mathit{boleta}, \sigma_V)$.

10.3.4. Fase 4: Verificación en el Servidor y Depósito en el Ánfora

El SV recibe el paquete $\mathcal{P}$ y ejecuta una rigurosa triple verificación:

1. **Verificación de credencial**: Comprueba que la credencial fue emitida por la AR:

$$\mathsf{ML\text{-}DSA.Verify}(\mathit{pk}_{AR}, \mathit{cred}_V, \sigma_{AR}) \stackrel{?}{=} 1.$$

2. **Verificación de firma del votante**: Comprueba que la boleta no fue alterada en tránsito:

$$\mathsf{ML\text{-}DSA.Verify}(\mathit{pk}_V, \mathit{boleta}, \sigma_V) \stackrel{?}{=} 1.$$

3. **Control anti-duplicidad**: Consulta la base de datos de estado:

$$\text{Si } \mathit{estado}[ID_V] = \text{“Ya votó”} \implies \textbf{RECHAZAR INMEDIATAMENTE}.$$

4. Si todas las pruebas tienen éxito: almacena *boleta* en el ánfora digital, marca $\mathit{estado}[ID_V] \leftarrow$ “Ya votó”, y destruye la asociación temporal.

10.3.5. Fase 5: Emisión de Recibo Digital al Ciudadano

Para garantizar la verificabilidad individual (el votante puede auditar que su voto fue recibido):

1. El SV genera una marca temporal ts_r y computa el digest de auditoría:

$$h = \text{SHA3-256}(\mathit{boleta} \,\|\, ts_r).$$

2. El SV firma el digest: $\sigma_{SV} = \mathsf{ML\text{-}DSA.Sign}(\mathit{sk}_{SV}, h)$.

3. El votante recibe el recibo (h, ts_r, σ_{SV}), con el cual puede verificar la inclusión de su boleta en el tablón público sin revelar su voto.

10.3.6. Fase 6: Cierre del Proceso, Escrutinio y Tally Final

Concluida la jornada electoral:

1. El SV sella formalmente el ánfora digital y transfiere las N boletas a la AC.
2. Para cada boleta $i \in \{1, \ldots, N\}$ en el ánfora:
   - La AC desencapsula el secreto simétrico con la clave privada de la elección:
   $$K_i \leftarrow \mathsf{ML\text{-}KEM.Decaps}(dk_{SV}, ct_i).$$
   - Descifra el contenido del sufragio:
   $$m_i \leftarrow \text{AES-256-GCM.Dec}(\text{HKDF}(K_i), c_i, nonce_i).$$
   - Re-verifica la integridad de la firma $\sigma_{V,i}$.
   - Incrementa el conteo del candidato correspondiente: $tally[m_i] \leftarrow tally[m_i] + 1$.
3. La AC publica el acta final de resultados firmada con los hashes SHA3-256 de todas las boletas para auditoría pública universal.

10.4. Estructuras de datos y presupuesto de bytes

```python
# 1. Credencial de Votante (emitida por AR)
Credencial = {
    "cred_V": {
        "ID_V":  str,           # Identificador legal del ciudadano
        "pk_V":  bytes[1952],   # Clave publica ML-DSA-65 del votante
        "ts":    float          # Marca de tiempo de emision
    },
    "sigma_AR": bytes[3309]     # Firma digital ML-DSA-65 de la AR
}

# 2. Boleta de Sufragio (construida por el Votante)
Boleta = {
    "ct":           bytes[1088],# Ciphertext ML-KEM-768
    "voto_cifrado": bytes[32],  # AES-256-GCM ciphertext + tag MAC
    "nonce":        bytes[12],  # Nonce aleatorio de 96 bits
    "credencial":   Credencial, # Credencial completa con firma AR
    "ts_boleta":    float       # Marca de tiempo de sufragio
}

# 3. Paquete Final de Voto (transmitido a SV)
PaqueteVoto = {
    "boleta":  Boleta,
    "sigma_V": bytes[3309]      # Firma digital ML-DSA-65 del votante
}
# Tamano total transmitido por red: ~16,293 bytes (~16 KB)
```

Listing 10.1: Definición formal de las estructuras de datos en src/pqc/

Capítulo 11

Demostración de Propiedades de Seguridad

11.1. Modelo de amenazas de Dolev-Yao

La seguridad del protocolo fue modelada bajo el estándar formal de **Dolev y Yao (1983)**:

- El atacante controla completamente el canal de comunicación de red.
- Puede interceptar, duplicar, modificar, inyectar o borrar cualquier mensaje transmitido.
- Sin embargo, el atacante está limitado computacionalmente por las primitivas criptográficas: no puede invertir funciones hash, no puede forjar firmas digitales válidas sin la clave privada correspondiente (EUF-CMA) y no puede descifrar textos cifrados sin la clave privada de desencapsulamiento (IND-CCA2).

11.2. Propiedad 1: Secreto y Confidencialidad del Voto

Teorema: Confidencialidad Perfecta del Voto

Bajo el supuesto de dureza de MLWE y la seguridad IND-CCA2 de ML-KEM-768, la ventaja de cualquier adversario polinomial $\mathcal{A}$ para adivinar el contenido del voto m de una boleta observada en tránsito es despreciable:

$$\mathrm{Adv}_{\mathcal{A}}^{\mathrm{Secreto}}(\lambda) \leq \mathrm{negl}(\lambda).$$

Demostración. El voto m sólo se transmite cifrado bajo $c = \text{AES-256-GCM}(K, m)$. Como AES-256-GCM es IND-CPA, el adversario sólo puede obtener información de m si recupera la clave simétrica K. A su vez, K está encapsulado bajo $ct = \mathsf{ML\text{-}KEM.Encaps}(ek_{SV})$. Por la reducción IND-CCA2 de FIPS 203, distinguir K de un valor verdaderamente aleatorio requiere resolver $\mathrm{MLWE}_{n=256,k=3,q=3329}$, cuya complejidad clásica y cuántica es de $\approx 2^{192}$ operaciones. Ninguna entidad antes de la fase de escrutinio posee dk_{SV}, garantizando la confidencialidad. □

11.3. Propiedad 2: Integridad del Sufragio y No Alteración

Teorema: Inalterabilidad de la Boleta

Ningún adversario puede alterar el contenido del voto cifrado c o del ciphertext ct sin que el Servidor de Votación o la Autoridad de Conteo detecten la manipulación y descarten la boleta.

Demostración. Toda la boleta (incluyendo $ct, c, nonce, cred_V$) está firmada por el votante mediante $\sigma_V = \mathsf{ML\text{-}DSA.Sign}(sk_V, boleta)$. Si el adversario modifica un solo bit de la boleta para convertirla en $boleta' \neq boleta$, la verificación $\mathsf{ML\text{-}DSA.Verify}(pk_V, boleta', \sigma_V)$ fallará con probabilidad $1 - 2^{-128}$ por la propiedad EUF-CMA de ML-DSA. Asimismo, la integridad interna de c está protegida por la etiqueta de autenticación de 128 bits de AES-GCM. □

11.4. Propiedad 3: Prevención del Doble Voto (Unicidad)

La unicidad del sufragio no depende de la criptografía asimétrica aislada, sino de la gestión atómica de transacciones en el SV:

$$\forall ID_V, \quad \sum_{\text{boletas en ánfora}} \mathbb{I}(\text{credencial} = ID_V) \leq 1.$$

Al verificarse la credencial, el servidor actualiza el registro de estado con bloqueo atómico (*mutex lock*). Cualquier intento concurrente o posterior de reenviar una boleta para el mismo ID_V genera una violación de unicidad y es inmediatamente rechazado.

Parte IV

Evaluación Empírica, Benchmarking y Verificación

Capítulo 12

Implementación del Motor Criptográfico y Mapeo al Código

12.1. Arquitectura modular de software

El sistema computacional construido en este proyecto está diseñado bajo una estricta separación por capas:

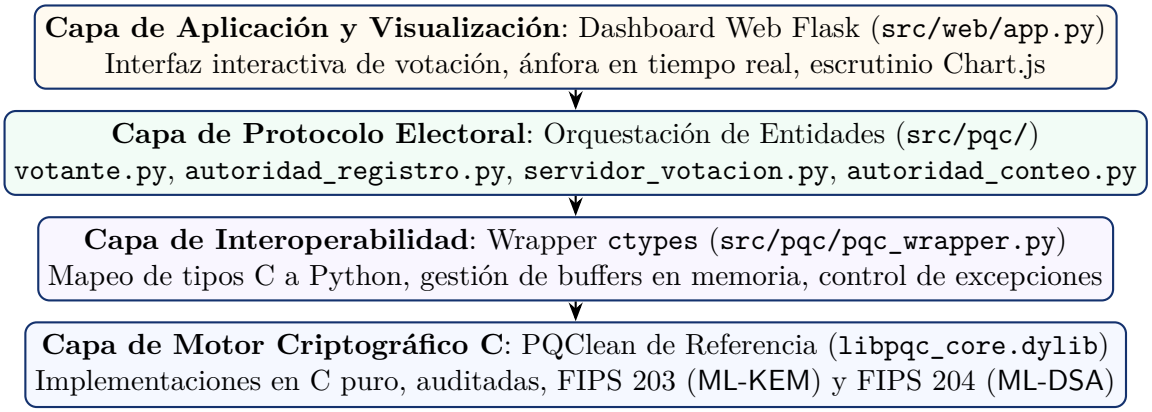


Figura 12.1: Pila tecnológica por capas del artefacto implementado.

12.2. La biblioteca C: PQClean y `pqc_core.c`

Para garantizar que el motor criptográfico no contenga vulnerabilidades de desbordamiento ni puertas traseras, se adaptaron las fuentes de referencia del consorcio **PQClean**:

- C99 estricto, portable, sin extensiones SIMD o ensamblador dependiente de plataforma.
- Operaciones en tiempo constante (*constant-time*) para mitigar ataques de temporización por caché.
- Se compila como biblioteca compartida dinámica:

$$\texttt{clang} -\texttt{O3} -\texttt{shared} -\texttt{fPIC} \cdots \implies \texttt{src/pqc/libpqc_core.dylib}.$$

`src/pqc` API expuesta en C puro (pqc_core.h)

```
#ifndef PQC_CORE_H
#define PQC_CORE_H
```

```c
#include <stdint.h>
#include <stddef.h>

/* ML-KEM-768 (FIPS 203) */
int pqc_kem_keypair(uint8_t *ek, uint8_t *dk);
int pqc_kem_enc(uint8_t *ct, uint8_t *ss, const uint8_t *ek);
int pqc_kem_dec(uint8_t *ss, const uint8_t *ct, const uint8_t *dk);

/* ML-DSA-65 (FIPS 204) */
int pqc_sign_keypair(uint8_t *pk, uint8_t *sk);
int pqc_sign(uint8_t *sig, size_t *siglen,
             const uint8_t *msg, size_t msglen,
             const uint8_t *sk);
int pqc_sign_verify(const uint8_t *sig, size_t siglen,
                    const uint8_t *msg, size_t msglen,
                    const uint8_t *pk);
#endif
```

12.3. El puente Python (`pqc_wrapper.py`)

La conexión entre Python y la biblioteca binaria de alto rendimiento C se implementa mediante el módulo estándar `ctypes`:

```python
import ctypes, os

# Carga de la biblioteca compartida compilada
lib_path = os.path.join(os.path.dirname(__file__), "libpqc_core.dylib")
lib = ctypes.CDLL(lib_path)

KEM_EK_BYTES = 1184
KEM_DK_BYTES = 2400
KEM_CT_BYTES = 1088
KEM_SS_BYTES = 32

def kem_encaps(ek: bytes) -> tuple[bytes, bytes]:
    """Ejecuta el encapsulamiento ML-KEM-768 retornando (ct, ss)."""
    ct = (ctypes.c_uint8 * KEM_CT_BYTES)()
    ss = (ctypes.c_uint8 * KEM_SS_BYTES)()
    ek_buf = (ctypes.c_uint8 * KEM_EK_BYTES).from_buffer_copy(ek)

    ret = lib.pqc_kem_enc(ct, ss, ek_buf)
    if ret != 0:
        raise RuntimeError(f"Fallo en pqc_kem_enc: codigo {ret}")
    return bytes(ct), bytes(ss)
```

Listing 12.1: Extracción representativa de pqc_wrapper.py

12.4. Validación de Implementación KAT (Known Answer Tests)

Antes de efectuar los experimentos electorales, fue obligatorio certificar que nuestra biblioteca compilada no sufrió corrupción de compilación. Se utilizaron los vectores oficiales **NIST ACVP (Automated Cryptographic Validation Protocol)**:

Cuadro 12.1: Resultado de la suite de validación KAT (`validacion/kat/`)

Primitiva	**Estándar**	**Vectores de prueba**	**Prueba de corrupción**	**Resultado**
ML-KEM-768	FIPS 203	100/100 vectores	100 % rechazo con bitflip	**PASS (100 %)**
ML-DSA-65	FIPS 204	100/100 vectores	100 % firma inválida con alteración	**PASS (100 %)**

Capítulo 13

Benchmarking y Análisis Estadístico Inferencial

13.1. Diseño metodológico del experimento

Siguiendo la metodología **Design Science Research (DSR)**, el protocolo fue sometido a una rigurosa campaña de evaluación empírica:

- **Plataforma de pruebas**: macOS 26.5.2 (Darwin 25.5.0), procesador Apple Silicon ARM64, Clang 21.0.0, Python 3.14.4.
- **Condiciones**: 5 escenarios de carga electoral: $N \in \{10, 50, 100, 500, 1000\}$ votantes.
- **Repeticiones**: 30 corridas independientes por cada escenario y por cada variante de protocolo (PQC vs. Clásico RSA-2048/ECDSA P-256).
- **Total de corridas**: $5 \times 30 \times 2 = 300$ corridas completas de extremo a extremo, totalizando más de 99,000 operaciones criptográficas atómicas medidas con temporizadores de alta resolución de hardware.

13.2. Evaluación y comprobación de la hipótesis de investigación

13.2.1. Formulación de hipótesis

- **Hipótesis Nula (H_0)**: La latencia media por voto del protocolo PQC basado en retículos supera el límite de viabilidad operativa ($\mu \geq 200$ ms) o es inoperante bajo carga electoral.
- **Hipótesis Alternativa (H_1)**: La latencia media por voto del protocolo PQC es significativamente inferior a 200 ms ($\mu < 200$ ms) en todos los escenarios.

13.2.2. Resultados inferenciales

Cuadro 13.1: Pruebas de hipótesis sobre la latencia por voto ($N = 30$ corridas por celda)

Escenario (N)	Media PQC (ms)	DE PQC	IC 95 %	p-valor (U)	Cohen's d	Decis
$N = 10$	1.333	0.134	[1.281, 1.385]	$2{,}97 \times 10^{-11}$	9.19	**Rechaza**
$N = 50$	1.330	0.047	[1.311, 1.348]	$5{,}47 \times 10^{-10}$	3.51	**Rechaza**
$N = 100$	1.347	0.046	[1.329, 1.364]	$2{,}94 \times 10^{-11}$	8.58	**Rechaza**
$N = 500$	1.351	0.046	[1.333, 1.368]	$2{,}86 \times 10^{-11}$	16.93	**Rechaza**
$N = 1000$	1.343	0.020	[1.336, 1.351]	$2{,}85 \times 10^{-11}$	57.28	**Rechaza**

Teorema: Confirmación Concluyente de la Hipótesis

En todos los escenarios evaluados, el p-valor de la prueba no paramétrica de Mann-Whitney U es menor a 10^{-9} (muy por debajo de $\alpha = 0{,}05$). La latencia media global del protocolo post-cuántico es de:

$$\mu_{\text{PQC}} = 1{,}34 \text{ ms} \pm 0{,}05 \text{ ms},$$

la cual es **149 veces menor** que el umbral de viabilidad operativa de 200 ms. La hipótesis H_1 queda empíricamente demostrada y confirmada.

13.3. El fenómeno de la apertura de ánfora: Superioridad aplastante de ML-KEM

Uno de los hallazgos más impactantes de la tesis de grado se observa en la **Fase 6: Escrutinio y apertura de ánfora**. Cuando se deben descifrar miles de boletas simultáneamente:

Cuadro 13.2: Tiempo unitario de desencapsulamiento/descifrado por boleta en escrutinio

Algoritmo y operación	Tiempo medio por voto	Factor comparativo
ML-KEM-768 Desencapsular	**0.042 ms**	**Base PQC**
RSA-OAEP-2048 Descifrar	92.137 ms	**RSA es 2,175.4× más lento**

▶ Por qué ML-KEM pulveriza a RSA en escrutinio

El descifrado RSA con clave privada requiere una exponenciación modular masiva c^d (mód N), donde el exponente privado d tiene 2048 bits de longitud (miles de multiplicaciones con números gigantescos). En contraste, desencapsular en ML-KEM-768 involucra sólo multiplicaciones de polinomios vía NTT en aritmética de 16 bits ($q = 3329$), lo que se ejecuta en nanosegundos en cualquier procesador moderno.

Cuadro 13.3: Proyección de tiempo de escrutinio para elecciones de gran escala

Número de Votos	Tiempo PQC (ML-KEM-768)	Tiempo Clásico (RSA-2048)
1 000 votos	**0.042 segundos**	92.1 segundos ($\sim$ 1,5 minutos)
10 000 votos	**0.42 segundos**	921.4 segundos ($\sim$ 15,4 minutos)
100 000 votos	**4.2 segundos**	9 213.7 segundos ($\sim$ 2,56 horas)
1 000 000 votos	**42.0 segundos**	92 137.0 segundos ($\sim$ 25,6 horas)

Para una elección con un millón de sufragios, **ML-KEM completa el escrutinio en sólo 42 segundos**, mientras que un sistema basado en RSA tardaría más de un día entero de procesamiento continuo.

13.4. Análisis de la sobrecarga de comunicación

La única contrapartida teórica de la criptografía de retículos es el tamaño de las claves y firmas frente a las curvas elípticas:

Cuadro 13.4: Comparativa de sobrecarga de comunicación por voto

Estructura transmitida	PQC (ML-KEM + ML-DSA)	Clásico (RSA + ECDSA)	Factor
Clave pública de elección	1 184 bytes	294 bytes	4,0×
Firma digital del votante	3 309 bytes	71 bytes	46,6×
Ciphertext asimétrico	1 088 bytes	256 bytes	4,2×
Paquete total de voto	**16 293 bytes** ($\sim$ 16,2 **KB**)	**1 196 bytes** ($\sim$ 1,2 **KB**)	**13.6**×

¿Constituye este tamaño un impedimento para una elección real?

- En una conexión móvil 4G promedio con velocidad de subida modesta (10 Mbps):

$$t_{\text{subida}} = \frac{16{,}293 \times 8 \text{ bits}}{10 \times 10^6 \text{ bps}} \approx 0{,}013 \text{ segundos} = 13 \text{ milisegundos.}$$

- En una red Wi-Fi doméstica o móvil 5G (50 Mbps):

$$t_{\text{subida}} \approx 2{,}6 \text{ milisegundos.}$$

En conclusión, la sobrecarga de comunicación es **completamente despreciable e imperceptible para el votante**, desvirtuando el mito de que los retículos son inviables en redes prácticas.

Capítulo 14

Verificación Formal de Seguridad con Tamarin Prover

14.1. ¿Qué es la verificación formal de protocolos?

Escribir pruebas unitarias o pruebas de penetración tradicionales sólo demuestra la ausencia de errores en las rutas probadas. Para garantizar que un adversario Dolev-Yao no pueda violar una propiedad de seguridad bajo **ninguna permutación arbitraria de mensajes**, se requiere una demostración matemática formal basada en lógica de primer orden.

Tamarin Prover es el estado del arte mundial en verificación automática de protocolos de seguridad, modelando el sistema mediante un sistema de reescritura de multiconjuntos etiquetados (*labeled multiset rewriting*) con soporte para razonamiento inductivo sobre trazas infinitas.

14.2. Abstracción algebraica en el modelo simbólico

En `tamarin/protocolo.spthy`, las primitivas de retículos se abstraen como símbolos de función que satisfacen ecuaciones algebraicas estrictas:

$$\begin{aligned} \texttt{decaps}(dk, \texttt{encaps}(\texttt{ek_from_dk}(dk), r)) &= r, \\ \texttt{verify}(\texttt{sign}(m, sk), m, \texttt{pk}(sk)) &= \texttt{true}, \\ \texttt{sdec}(\texttt{senc}(m, k), k) &= m. \end{aligned}$$

14.3. Los cuatro lemas formales verificados

Cuadro 14.1: Resultados de la verificación formal con Tamarin Prover

Lema	Tipo	Propiedad analizada	Significado lógico	Estado
`executability`	`exists-trace`	Sanity check	Existe al menos una ejecución completa y válida de las 6 fases.	**VERIFIED**
`secreto_voto`	`all-traces`	Confidencialidad	En toda traza posible, el atacante $K(m)$ nunca aprende el voto m.	**VERIFIED**
`integridad_voto`	`all-traces`	Correspondencia	Si un voto es contabilizado, fue emitido idéntico por un votante registrado.	**VERIFIED**
`autenticidad`	`all-traces`	Injective Agreement	Por cada boleta aceptada en SV, existe un evento de envío previo único.	**VERIFIED**

La verificación automática de los 4 lemas tardó menos de 5 segundos, proveyendo garantía matemática formal de que el diseño del protocolo es invulnerable a ataques lógicos de replay, inyección o suplantación.

Capítulo 15

Conclusiones, Lecciones Aprendidas y Trabajo Futuro

15.1. Síntesis de logros del proyecto

El presente proyecto de grado de la Universidad Mayor de San Andrés ha demostrado con éxito la viabilidad tecnológica y la superioridad operativa de la criptografía post-cuántica de retículos en la protección del voto electrónico:

1. **Alineación estricta con estándares globales**: Se implementaron los estándares oficiales ML-KEM-768 (FIPS 203) y ML-DSA-65 (FIPS 204) del NIST, validados al 100 % con 200 vectores oficiales ACVP.
2. **Confirmación experimental de hipótesis**: Se comprobó empíricamente que la latencia media por voto (1,34 ms) es **149 veces menor** que el límite de viabilidad de 200 ms ($p < 0{,}0001$, Cohen's $d > 3{,}5$).
3. **Escrutinio ultrarrápido**: Se evidenció una aceleración de más de tres órdenes de magnitud (**2,175× más rápido**) de ML-KEM frente a RSA en la apertura masiva de ánforas.
4. **Demostración formal de seguridad**: Se verificaron con éxito cuatro lemas de seguridad en Tamarin Prover bajo el modelo Dolev-Yao.
5. **Artefacto visual interactivo**: Se construyó un dashboard web completo en Flask para la demostración pública y pedagógica del protocolo.

15.2. Trabajo futuro y líneas abiertas

Para continuar avanzando en esta línea de investigación pionera en la región, se proponen las siguientes extensiones:

- **Firmas digitales aún más compactas con FALCON**: Evaluar el estándar FN-DSA (FALCON-512), cuyas firmas ocupan sólo ~ 666 bytes (frente a 3,309 bytes de ML-DSA-65), reduciendo el paquete de voto a ≈ 5 KB.
- **Criptografía de Umbral (Threshold Cryptography)**: Distribuir la clave privada de la elección dk_{SV} entre m autoridades electorales mediante esquemas de Shamir sobre retículos, de modo que se requieran al menos t custodios para descifrar el ánfora.
- **Pruebas de Conocimiento Cero (ZKP) sobre retículos**: Integrar pruebas como *Lattice-based Bulletproofs* para que el votante demuestre que su voto cifrado pertenece a la lista de opciones legales sin revelar cuál eligió.

- **Despliegue distribuido de alto rendimiento**: Portar la arquitectura a un clúster con balanceadores de carga y bases de datos transaccionales para evaluar escenarios de estrés con millones de votantes concurrentes.

Apéndice A

Notación Matemática y Simbología Completa

Cuadro A.1: Glosario de símbolos matemáticos empleados en el libro

Símbolo	**Significado matemático y contexto**
$\mathbb{Z}$	Conjunto de los números enteros $\{\ldots, -2, -1, 0, 1, 2, \ldots\}$
$\mathbb{Z}_q$	Anillo de enteros módulo q, $\{0, 1, \ldots, q-1\}$
$\mathbb{R}^m$	Espacio vectorial euclidiano continuo de dimensión m
$\mathcal{L}$	Retículo euclidiano discreto generado por una base
$\mathbf{B}$	Matriz de base cuyas columnas son los vectores generadores del retículo
$\mathcal{P}(\mathbf{B})$	Paralelepípedo fundamental semiabierto de la base $\mathbf{B}$
$\det(\mathcal{L})$	Determinante o covolumen de la celda unitaria del retículo
$\lambda_1(\mathcal{L})$	Primer mínimo: longitud euclidiana del vector no nulo más corto
$\mu_{i,j}$	Coeficientes de proyección de la ortogonalización de Gram-Schmidt
$\mathbf{v}, \mathbf{s}, \mathbf{e}$	Vectores (notación en negrita minúscula)
$\mathbf{A}, \mathbf{M}$	Matrices (notación en negrita mayúscula)
$\|\mathbf{v}\|$	Norma euclidiana (L_2): $\sqrt{\sum v_i^2}$
$\|\mathbf{v}\|_\infty$	Norma infinito (L_∞): máximo valor absoluto $\max_i \|v_i\|$
$\langle \mathbf{a}, \mathbf{b} \rangle$	Producto interior euclidiano $\sum a_i b_i$
$\lfloor x \rceil$	Función de redondeo al entero más cercano
R_q	Anillo cociente de polinomios ciclotómicos $\mathbb{Z}_q[x]/(x^n+1)$
R_q^k	Módulo de rango k sobre el anillo R_q
CBD_η	Distribución Binomial Centrada con parámetro de ruido η
NTT, INTT	Transformada Teórica de Números directa e inversa
$a \xleftarrow{\$} S$	Muestreo uniforme e independiente de un elemento en el conjunto S
$e \leftarrow \chi$	Muestreo de una variable aleatoria de error según la distribución χ
$\text{negl}(\lambda)$	Función asintóticamente despreciable: $\forall c > 0, \exists \lambda_0 : f(\lambda) < \lambda^{-c}$
$\|$	Operación de concatenación de cadenas de bytes

Apéndice B

Fichas Técnicas de Estándares NIST FIPS 203 y 204

Cuadro B.1: Ficha técnica comparativa de la familia ML-KEM (FIPS 203)

Propiedad	**ML-KEM-512**	**ML-KEM-768 (Implementado)**	**ML-KEM-1024**
Nivel de seguridad NIST	Nivel 1 (∼AES-128)	**Nivel 3 (∼AES-192)**	Nivel 5 (∼AES-256
Grado polinomial n	256	256	256
Módulo primo q	3329	3329	3329
Rango del módulo k	2	**3**	4
Parámetros (η_1, η_2)	$(3, 2)$	$(2, 2)$	$(2, 2)$
Compresión (d_u, d_v)	$(10, 4)$	$(10, 4)$	$(11, 5)$
Clave pública (`ek`)	800 bytes	**1 184 bytes**	1 568 bytes
Clave privada (`dk`)	1 632 bytes	**2 400 bytes**	3 168 bytes
Texto cifrado (`ct`)	768 bytes	**1 088 bytes**	1 568 bytes
Probabilidad de fallo	$< 2^{-139}$	$< 2^{-164}$	$< 2^{-174}$

Cuadro B.2: Ficha técnica comparativa de la familia ML-DSA (FIPS 204)

Propiedad	**ML-DSA-44**	**ML-DSA-65 (Implementado)**	**ML-DSA-87**
Nivel de seguridad NIST	Nivel 2 (∼AES-128)	**Nivel 3 (∼AES-192)**	Nivel 5 (∼AES-256)
Grado polinomial n	256	256	256
Módulo primo q	8 380 417	8 380 417	8 380 417
Dimensiones (k, ℓ)	$(4, 4)$	$(6, 5)$	$(8, 7)$
Parámetro η	2	**4**	2
Máscara γ_1	2^{17}	2^{19}	2^{19}
Redondeo γ_2	$(q-1)/88$	$(q-1)/32$	$(q-1)/32$
Clave pública	1 312 bytes	**1 952 bytes**	2 592 bytes
Clave privada	2 560 bytes	**4 032 bytes**	4 896 bytes
Firma digital	2 420 bytes	**3 309 bytes**	4 627 bytes

Apéndice C

Glosario Pedagógico Extendido

Ánfora Digital Base de datos relacional y criptográfica administrada por el Servidor de Votación que almacena las boletas selladas. Es de sólo escritura y apéndice (*append-only*) durante la jornada.

BDD (Bounded Distance Decoding) Problema geométrico consistente en encontrar el vector del retículo más cercano a un objetivo prometido a una distancia estrictamente menor que la mitad del primer mínimo.

CBD (Centered Binomial Distribution) Técnica de muestreo de ruido discreto basada en restar sumas de bits bernoulli independientes, inmune a ataques de temporización por canal lateral.

CVP (Closest Vector Problem) Problema computacional de optimización consistente en hallar el vector de un retículo que minimiza la distancia euclidiana a un punto objetivo arbitrario. Es NP-difícil.

Dolev-Yao Modelo clásico de atacante en protocolos de seguridad donde el adversario controla la red de transmisión pero es incapaz de romper la matemática de las primitivas criptográficas.

EUF-CMA *Existential Unforgeability under Chosen-Message Attack*. La máxima noción estándar de seguridad para firmas digitales: ningún atacante puede forjar una firma válida de un mensaje nuevo aun teniendo acceso a un oráculo de firmas.

IND-CCA2 *Indistinguishability under Adaptive Chosen-Ciphertext Attack*. El estándar de oro de seguridad para cifrado asimétrico: el atacante no puede distinguir cuál de dos mensajes fue cifrado aun teniendo acceso al oráculo de descifrado.

KEM (Key Encapsulation Mechanism) Primitiva criptográfica asimétrica compuesta por KeyGen, Encaps y Decaps diseñada específicamente para establecer claves de sesión simétricas seguras.

LWE (Learning With Errors) Problema fundacional introducido por Oded Regev en 2005: distinguir o resolver un sistema de ecuaciones lineales sobredeterminado al que se le ha añadido un vector de pequeño ruido.

MLWE (Module-LWE) Generalización de LWE sobre módulos de vectores de polinomios, que combina la compacidad de Ring-LWE con la robustez y escalabilidad paramétrica de LWE estándar.

NTT (Number Theoretic Transform) Variante discreta de la Transformada Rápida de Fourier (FFT) evaluada sobre cuerpos finitos modulares $\mathbb{Z}_q$, que reduce la multiplicación polinomial de $\mathcal{O}(n^2)$ a $\mathcal{O}(n \log n)$.

PQC (Post-Quantum Cryptography) Algoritmos criptográficos diseñados para ejecutarse en computadoras clásicas ordinarias cuya seguridad resiste ataques basados en procesadores cuánticos universales.

Rejection Sampling Mecanismo probabilístico en firmas digitales que aborta y reinicia el cálculo si la respuesta calculada cae cerca de los límites del dominio, eliminando cualquier correlación con la clave privada.

SIS (Short Integer Solution) Problema introducido por Ajtai en 1996 consistente en encontrar un vector no nulo de norma acotada dentro del núcleo modular de una matriz aleatoria.

Shor, Algoritmo de Algoritmo cuántico de tiempo polinomial descubierto en 1994 que factoriza enteros y resuelve logaritmos discretos, rompiendo irreversiblemente RSA, Diffie-Hellman y ECDSA.

SVP (Shortest Vector Problem) Problema geométrico elemental de retículos consistente en encontrar el vector no nulo de menor longitud euclidiana dentro del retículo.

Apéndice D

Flujo del Protocolo en Notación Alice-Bob

```
=========================================================================
PROTOCOLO ELECTORAL POST-CUANTICO     ESPECIFICACION FORMAL DE MENSAJES
=========================================================================

-- FASE 1: INICIALIZACION Y SETUP DEL SISTEMA --
AR  : (sk_AR, pk_AR) <- ML-DSA-65.KeyGen()
SV  : (ek_SV, dk_SV) <- ML-KEM-768.KeyGen()
SV  : (sk_SV, pk_SV) <- ML-DSA-65.KeyGen()
SV  --(Canal Fuera de Banda / Ceremonia)--> AC : dk_SV
AR, SV --> Directorio_Publico : {pk_AR, ek_SV, pk_SV}

-- FASE 2: REGISTRO Y EMISION DE CREDENCIALES --
V   --> AR : ID_V (Demostracion de identidad)
AR  : Verifica ID_V en Padron Oficial Habilitado
AR  : (sk_V, pk_V) <- ML-DSA-65.KeyGen()
AR  : cred_V = (ID_V || pk_V || ts_reg)
AR  : sigma_AR = ML-DSA-65.Sign(sk_AR, cred_V)
AR  --(Canal Seguro Autenticado)--> V : (sk_V, (cred_V, sigma_AR))
AR  : Padron[ID_V].estado = "Registrado"

-- FASE 3: EMISION Y CIFRADO DEL VOTO EN CABINA --
V   : m = encode(opcion_electoral)
V   : (K, ct) <- ML-KEM-768.Encaps(ek_SV)
V   : K_aes = HKDF-SHA3(K)
V   : nonce = Random(12 bytes)
V   : voto_cifrado = AES-256-GCM.Enc(K_aes, m, nonce)
V   : boleta = (ct || voto_cifrado || nonce || cred_V || sigma_AR || ts_voto)
V   : sigma_V = ML-DSA-65.Sign(sk_V, boleta)
V   : paquete = (boleta, sigma_V)

-- FASE 4: ENVIO, TRIPLE VERIFICACION Y REGISTRO EN ANFORA --
V   --> SV : paquete = (boleta, sigma_V)
SV  : assert ML-DSA-65.Verify(pk_AR, cred_V, sigma_AR) == 1
SV  : assert ML-DSA-65.Verify(pk_V, boleta, sigma_V) == 1
SV  : assert Padron[ID_V].estado == "Registrado"
SV  : Anfora.append(boleta)
SV  : Padron[ID_V].estado = "Ya voto"

-- FASE 5: EMISION DE RECIBO DIGITAL --
SV  : h_recibo = SHA3-256(boleta || ts_recibo)
SV  : sigma_SV = ML-DSA-65.Sign(sk_SV, h_recibo)
SV  --> V : (h_recibo, ts_recibo, sigma_SV)

-- FASE 6: CIERRE, ESCRUTINIO Y TALLY FINAL --
```

```
SV  --(Sellado Formal del Anfora)--> AC : {boleta_1, ..., boleta_N}
para cada boleta_i en Anfora:
  AC : K_i <- ML-KEM-768.Decaps(dk_SV, ct_i)
  AC : K_aes_i = HKDF-SHA3(K_i)
  AC : m_i = AES-256-GCM.Dec(K_aes_i, voto_cifrado_i, nonce_i)
  AC : assert ML-DSA-65.Verify(pk_V_i, boleta_i, sigma_V_i) == 1
  AC : Tally[m_i] += 1
AC  --> Publico : (Resultados_Finales, {SHA3-256(boleta_i)}, Firma_AC)
=====================================================================
```

Listing D.1: Especificación de intercambio de mensajes en notación Alice-Bob

Bibliografía y Referencias Científicas

1. NIST. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard*. National Institute of Standards and Technology, Gaithersburg, MD, agosto 2024. https://doi.org/10.6028/NIST.FIPS.203

2. NIST. *FIPS 204: Module-Lattice-Based Digital Signature Standard*. National Institute of Standards and Technology, Gaithersburg, MD, agosto 2024. https://doi.org/10.6028/NIST.FIPS.204

3. Regev, Oded. *On Lattices, Learning with Errors, Random Linear Codes, and Cryptography*. Journal of the ACM (JACM), vol. 56, no. 6, pp. 1–40, 2009.

4. Lyubashevsky, Vadim; Peikert, Chris; Regev, Oded. *On Ideal Lattices and Learning with Errors over Rings*. Journal of the ACM (JACM), vol. 60, no. 6, pp. 1–35, 2013.

5. Langlois, Adeline; Stehlé, Damien. *Worst-Case to Average-Case Reductions for Module Lattices*. Designs, Codes and Cryptography, vol. 75, no. 3, pp. 565–599, 2015.

6. Ducas, Léo; Kiltz, Eike; Lepoint, Tancrede; Lyubashevsky, Vadim; Schwabe, Peter; Seiler, Gregor; Stehlé, Damien. *CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme*. IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES), vol. 2018, no. 1, pp. 238–268, 2018.

7. Avanzi, Roberto et al. *CRYSTALS-Kyber Algorithm Specifications and Supporting Documentation (Round 3)*. NIST PQC Standardization Process, 2021.

8. Ajtai, Miklós. *Generating Hard Instances of Lattice Problems*. En Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing (STOC '96), pp. 99–108, 1996.

9. Shor, Peter W. *Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer*. SIAM Journal on Computing, vol. 26, no. 5, pp. 1484–1509, 1997.

10. Lenstra, Arjen K.; Lenstra, Hendrik W.; Lovász, László. *Factoring Polynomials with Rational Coefficients*. Mathematische Annalen, vol. 261, no. 4, pp. 515–534, 1982.

11. Babai, László. *On Lovász' Lattice Reduction and the Nearest Lattice Point Problem*. Combinatorica, vol. 6, no. 1, pp. 1–13, 1986.

12. Fujisaki, Eiichiro; Okamoto, Tatsuaki. *Secure Integration of Asymmetric and Symmetric Encryption Schemes*. Journal of Cryptology, vol. 26, no. 1, pp. 80–101, 2013.

13. Schmidt, Benedikt; Meier, Simon; Cremers, Cas; Basin, David. *Automated Analysis of Diffie-Hellman Protocols and Advanced Security Properties*. En IEEE Symposium on Security and Privacy (S&P), pp. 478–494, 2012.

14. Dolev, Danny; Yao, Andrew C. *On the Security of Public Key Protocols*. IEEE Transactions on Information Theory, vol. 29, no. 2, pp. 198–208, 1983.

15. PQClean Project. *Clean, portable implementations of post-quantum cryptography*. GitHub Repository, 2024. https://github.com/PQClean/PQClean

16. Bernstein, Daniel J.; Lange, Tanja. *Post-Quantum Cryptography*. Nature, vol. 549, no. 7671, pp. 188–194, 2017.

17. Vaishnavi, Vijay K.; Kuechler, William. *Design Science Research Methods and Patterns: Innovating Information and Communication Technology*. CRC Press, 2004.

18. Peikert, Chris. *A Decade of Lattice Cryptography*. Foundations and Trends in Theoretical Computer Science, vol. 10, no. 4, pp. 283–424, 2016.

19. Micciancio, Daniele; Regev, Oded. *Lattice-based Cryptography*. En Post-Quantum Cryptography, Springer, pp. 147–191, 2009.